

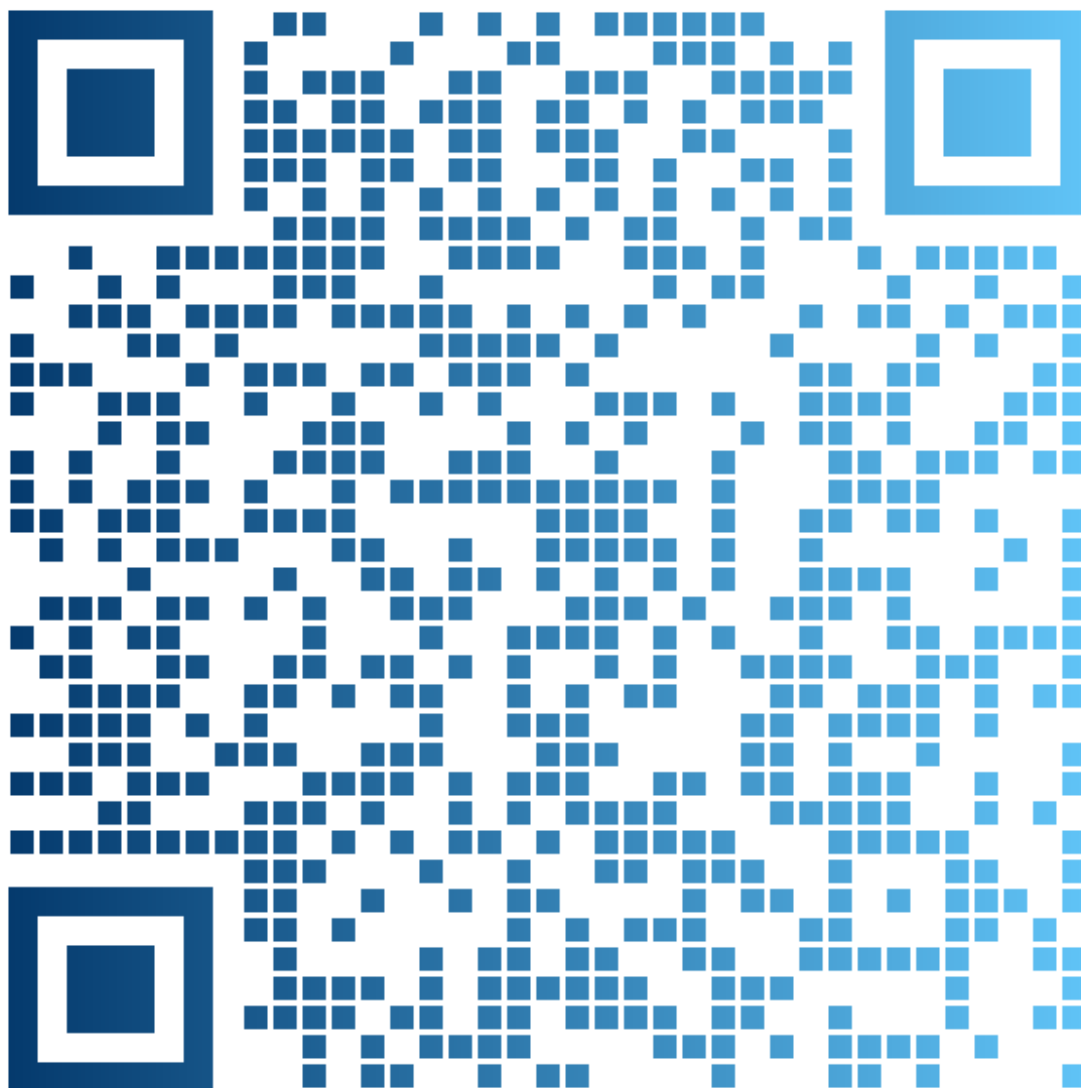


المدرسة العليا للتكنولوجيا
جامعة الحسن الثاني الدار البيضاء
Ecole Supérieure de Technologie
Université Hassan II de Casablanca

Analyse et visualisation de données

IAOUSSE M'barek

2026



Ce **manuel** a été élaboré pour servir de support de cours principal pour l'élément *Analyse et visualisation de données* dispensé dans le cadre de la Licence Professionnelle DBT-DAIA (*Développement d'Applications et Intelligence Artificielle*) en Temps Aménagé à l'École Supérieure de Technologie de Casablanca (ESTC).

L'auteur : M'barek IAOUSSE, Enseignant-Chercheur à l'EST Casablanca, Université Hassan II de Casablanca.



Sauf indications contraires, le contenu de ce manuel électronique est disponible en vertu des termes de la [Licence Creative Commons Attribution 4.0 International](https://creativecommons.org/licenses/by/4.0/).

Autorisations accordées :

Partager – copier, distribuer et communiquer le matériel par tous moyens et sous tous formats.

Adapter – remixer, transformer et créer à partir du matériel pour toute utilisation, y compris commerciale.

Sous réserve de citer l'auteur.

© M'barek IAOUSSE (2026)

Pour citer ce manuel : M'barek IAOUSSE. (2026). **Analyse et visualisation de données**. École Supérieure de Technologie de Casablanca. Sous licence CC BY 4.0.

Préface

Le présent manuel est le support de cours officiel de l'élément **Analyse et Visualisation des Données**, dispensé dans le cadre d'une formation en Développement d'Applications et Intelligence Artificielle.

La communauté étudiante visée par ce module provient de parcours techniques variés (DUT, BTS, DTS ou diplômes équivalents), ayant déjà acquis un bagage de base en programmation Python. La familiarité préalable avec la Data Science, l'analyse de données ou le Machine Learning n'est pas un prérequis : ce manuel se concentre volontairement sur ce qui précède et accompagne la modélisation — comprendre, préparer, explorer et représenter une donnée — sans aborder les algorithmes d'apprentissage automatique, qui font l'objet d'un module distinct.

Ce manuel vise à outiller les étudiants d'une **démarche méthodique** face à un jeu de données : savoir le questionner avant de le modéliser, savoir le nettoyer avant de l'analyser, et savoir le représenter avant de le communiquer. Il introduit progressivement les statistiques descriptives nécessaires, l'écosystème Python (**pandas**, **numpy**, **matplotlib**, **seaborn**), les principes d'un entrepôt de données, puis la construction d'une solution de Business Intelligence avec Power BI et Taipy. Un cas pratique final relie ces notions dans un même projet.

Prérequis

Le contenu de ce manuel est organisé pour des personnes ayant déjà une base et une expérience avec :

- la programmation en langage Python (variables, boucles, fonctions, structures de données de base) ;
- la manipulation élémentaire de fichiers et de tableaux.

Aucune connaissance préalable en analyse de données, en statistique ou en data visualization n'est nécessaire : les indicateurs utiles sont introduits et illustrés progressivement.

Livres et ressources complémentaires

Ce manuel s'inspire des ouvrages de référence en analyse de données et en visualisation, notamment *Python for Data Analysis* (McKinney 2022), consacré à la manipulation de données avec **pandas**, ainsi que *Storytelling with Data* (Knaflic 2015), référence sur la communication visuelle de résultats analytiques.

L'ensemble des exemples et exercices pratiques est développé en utilisant **Python** comme langage principal, avec **pandas** et **numpy** pour la manipulation des données, et **matplotlib/seaborn** pour la visualisation statique.

Note

Les bibliothèques scientifiques Python évoluant rapidement, veillez à utiliser un environnement virtuel basé sur Python 3.10 ou supérieur pour garantir la compatibilité des extraits de code présentés dans ce manuel.

Table des matières

Préface	i
Introduction à l'analyse de données	1
1. Qu'est-ce que l'analyse de données ?	1
Définition et Positionnement	1
Les quatre types d'analyse de données	2
2. Les premiers indicateurs à connaître	3
Compter et comparer	4
Résumer une variable numérique	4
3. Cycle de vie et mise en pratique avec Python	8
Le cycle de vie d'un projet d'analyse de données	8
Écosystème d'outils Python pour l'analyse de données	8
Exercices	13
Préparation des données	15
1. Collecte de données	15
1.1. Les bases de données	15
1.2. Les fichiers plats et formats de données	16
1.3. Les API	16
1.4. Le <i>web scraping</i>	17
1.5. Open data et capteurs / IoT	17
2. Nettoyage des données	17
2.1. Les valeurs manquantes	17
2.2. Les valeurs aberrantes (<i>outliers</i>)	18
2.3. Doublons et incohérences	19
2.4. Vers une démarche de qualité des données	19
3. Transformation des données	19
3.1. Encodage des variables catégorielles	19
3.2. Normalisation et standardisation	20
3.3. Discrétisation (<i>binning</i>)	20
3.4. Quelques transformations usuelles supplémentaires	20
Exercices	21
Exploration des données	22
1. Statistiques descriptives	22
1.1. Tendances centrale	22
1.2. Dispersion	23
1.3. Forme de la distribution	23
2. Analyse univariée, bivariée et multivariée	23
3. Corrélations et associations	24
3.1. Variables numériques : Pearson et Spearman	24
3.2. Variables catégorielles : tableaux croisés et V de Cramér	24
4. Agrégation : groupby et tableaux croisés dynamiques	25
5. Choisir une visualisation simple	25
6. Profilage automatique des données	26
Exercices	26

Introduction à l'entrepôt de données	28
1. Pourquoi un entrepôt de données ?	28
1.1. Les systèmes opérationnels	28
1.2. La base analytique	28
1.3. Entrepôt, lac et datamart	29
2. Le modèle en étoile	29
2.1. La table de faits	29
2.2. Les tables de dimensions	29
2.3. Exemple de modèle de ventes	30
3. Les mesures et les indicateurs	30
4. Le flux ETL/ELT	31
5. Gouvernance et sécurité, sans complexité inutile	31
Exercices	32
Business Intelligence avec Power BI et Taipy	33
1. Comprendre la BI	33
1.1. Du besoin à l'indicateur	33
1.2. Le parcours d'une solution BI	33
1.3. Règles d'un tableau de bord débutant	34
2. Power BI : une démarche structurée	34
2.1. Étape 1 — Importer les sources	34
2.2. Étape 2 — Transformer avec Power Query	34
2.3. Étape 3 — Construire le modèle	35
2.4. Étape 4 — Créer des mesures en DAX	35
2.5. Étape 5 — Construire les visuels	35
2.6. Étape 6 — Vérifier et partager	36
3. Taipy : créer une application BI en Python	36
3.1. Quand choisir Taipy ?	36
3.2. Un exemple Taipy minimal	36
3.3. Organiser une petite application	37
4. Comparaison synthétique	37
Exercices	38
Cas pratique complet : analyser les ventes d'une petite chaîne de magasins	39
1. Le besoin métier	39
1.1. Périmètre et définitions	39
2. Le jeu de données	40
2.1. Première lecture	40
3. Préparer les données	40
3.1. Dictionnaire minimal	40
3.2. Transformation	41
4. Calculer les indicateurs	41
4.1. Montants ligne par ligne	41
4.2. Par mois	42
4.3. Par magasin et par catégorie	42
4.4. Moyenne et médiane des montants de lignes	42
5. Construire le modèle analytique	43
6. Concevoir la page Power BI	43
7. Construire une vue Taipy	43
8. Interpréter et communiquer	44
Exercices	44
References	46

Introduction à l'analyse de données

💡 Objectifs du chapitre

À la fin de ce chapitre introductif, l'étudiant sera capable de :

- **Définir** l'analyse de données et la **situer** par rapport à la Data Science, la Business Intelligence (BI) et le Machine Learning ;
- **Distinguer** les quatre types d'analyse : descriptive, diagnostique, prédictive et prescriptive ;
- **Identifier** les grandes étapes du cycle de vie d'un projet d'analyse de données ;
- **Identifier** les outils de base utilisés dans ce cours.

1. Qu'est-ce que l'analyse de données ?

Définition et Positionnement

L'**analyse de données** (*Data Analysis*) désigne l'ensemble des démarches permettant d'inspecter, de nettoyer, de transformer et de modéliser statistiquement des données, dans le but d'en extraire des informations utiles, de soutenir une décision ou de valider une hypothèse.

💡 Analyse de données

Définition 0.0.1. L'analyse de données consiste à transformer des données brutes, souvent volumineuses et désorganisées, en **informations exploitables**, à travers une succession d'étapes de collecte, de nettoyage, d'exploration et de représentation.

Contrairement à ce que l'on pense parfois, l'essentiel du travail d'un analyste ne consiste pas à produire des graphiques, mais à **comprendre les données** : d'où elles viennent, ce qu'elles représentent réellement, quelles sont leurs limites et leurs biais. La visualisation n'intervient qu'en bout de chaîne, pour communiquer un résultat déjà établi.

Il existe quatre domaines liés à l'analyse de données : Data Analysis, Data Science, Business Intelligence, et Machine Learning. Ces quatre termes sont souvent confondus. Ils partagent une matière première commune (la donnée) mais diffèrent par leurs objectifs et leurs méthodes.

TABLE 1 – Quatre disciplines complémentaires autour de la donnée

Domaine	Question typique	Résultat produit
Business Intelligence (BI)	« Que s'est-il passé, et où en sommes-nous ? »	Tableaux de bord, indicateurs (KPI) suivis dans le temps
Analyse de données (Data Analysis)	« Pourquoi cela s'est-il produit ? Que peut-on en conclure ? »	Rapports, visualisations, recommandations argumentées
Data Science	« Que va-t-il probablement se passer ? »	Modèles statistiques et modèles de Machine Learning

Domaine	Question typique	Résultat produit
Machine Learning	« Comment automatiser une prédiction ou une décision à grande échelle ? »	Un modèle entraîné, déployé en production

⚠ Périimètre de ce cours

Ce module se concentre sur les deux premières colonnes de la Table 1 : la **préparation**, l'**exploration** et la **visualisation** des données. Les algorithmes de Machine Learning (régression, classification, clustering...) ne sont **pas traités ici** : ils font l'objet du cours [Machine Learning](#). Une bonne partie du travail présenté dans ce cours reste néanmoins un prérequis indispensable à tout projet de Machine Learning.

En pratique, ces frontières sont poreuses : un même professionnel peut, selon les entreprises, être appelé *Data Analyst* et exercer des missions de *Data Scientist*, ou inversement. Ce qui compte est de comprendre **la nature de la question posée** plutôt que le titre du poste.

Les quatre types d'analyse de données

On distingue classiquement quatre niveaux d'analyse, de la simple observation du passé jusqu'à la recommandation d'action, chacun s'appuyant sur le précédent.

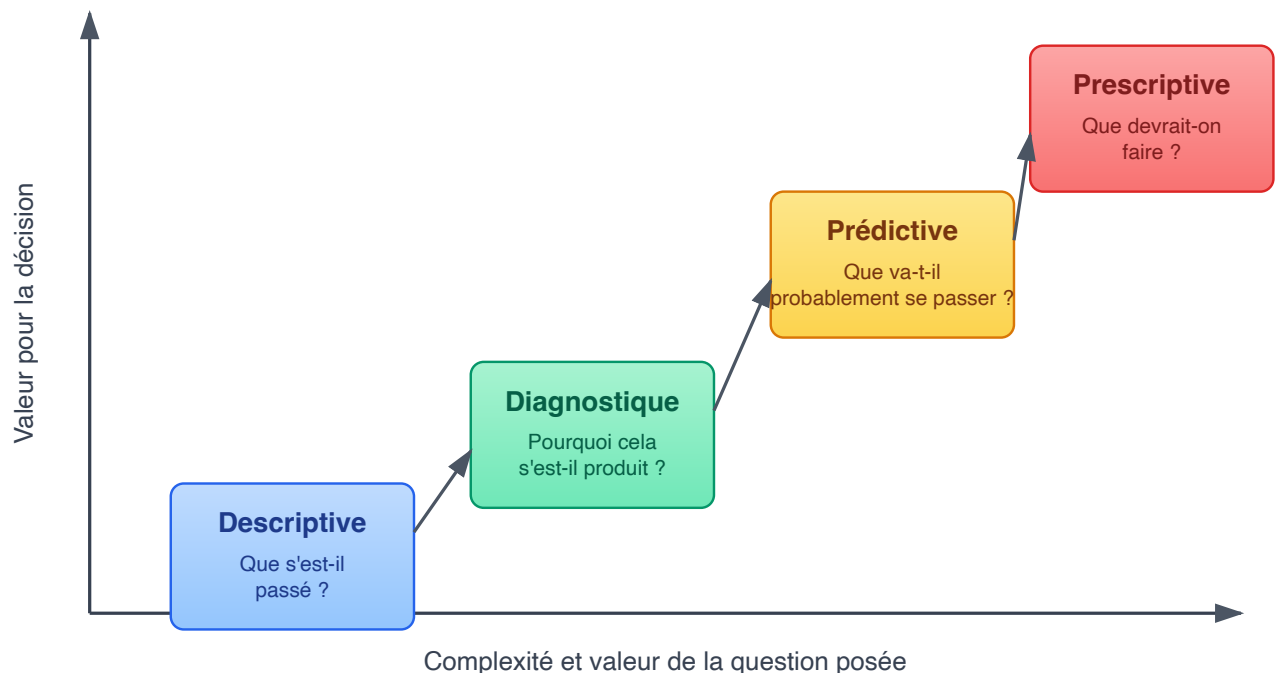


FIGURE 1 – Les quatre types d'analyse de données

TABLE 2 – Les quatre types d’analyse de données

Type d’analyse	Question posée	Exemple
Descriptive	Que s’est-il passé ?	Le chiffre d’affaires du mois dernier par région
Diagnostic	Pourquoi cela s’est-il passé ?	Pourquoi les ventes ont-elles chuté en mars dans le nord ?
Prédictive	Que va-t-il probablement se passer ?	Quel sera le chiffre d’affaires du mois prochain ?
Prescriptive	Que devrait-on faire ?	Quelle action marketing maximiserait les ventes du trimestre ?

💡 Exemple : une chaîne de supermarchés marocaine

- **Descriptive** : « Les ventes de la région de Casablanca ont baissé de 12 % en mars par rapport à février. »
- **Diagnostic** : en croisant les ventes avec le calendrier et la météo, on découvre que la baisse coïncide avec des travaux routiers ayant réduit l’affluence dans deux magasins.
- **Prédictive** (*hors périmètre de ce cours, traité en Machine Learning*) : un modèle estime les ventes attendues pour le mois suivant compte tenu de la reprise du trafic.
- **Prescriptive** : sur la base du diagnostic, l’équipe décide de renforcer temporairement les promotions dans les magasins concernés.

Ce cours se concentre principalement sur les analyses **descriptive** et **diagnostic** : elles reposent entièrement sur les techniques de préparation, d’exploration et de visualisation présentées dans les chapitres suivants, et ne nécessitent pas d’algorithme d’apprentissage automatique.

2. Les premiers indicateurs à connaître

Avant de construire un graphique ou un tableau de bord, il faut savoir **décrire simplement les données**.

Une **observation** est une ligne du tableau (par exemple une commande) et une **variable** est une colonne (par exemple le montant de la commande).

Une variable peut être de trois natures :

- **qualitative** : une catégorie ou une étiquette (ville, moyen de paiement, segment client...);
- **quantitative** : un nombre sur lequel on peut calculer (prix, quantité, durée, chiffre d’affaires...);
- **temporelle** : une date ou une période que l’on peut ordonner dans le temps (jour, mois, trimestre...).

Ces distinctions sont fondamentales : les indicateurs et les graphiques adaptés ne sont pas les mêmes selon le type de variable.

Compter et comparer

💡 Effectif et fréquence

L'**effectif** d'une catégorie est le nombre d'observations qui appartiennent à cette catégorie.
La **fréquence** (ou proportion) est la part que représente cet effectif dans l'ensemble des observations.

$$\text{fréquence} = \frac{\text{effectif de la catégorie}}{\text{effectif total}}$$

On l'exprime souvent en pourcentage en multipliant par 100.

Exemple – Effectif et fréquence

Sur 120 commandes, 30 proviennent de Casablanca :

$$\text{fréquence} = \frac{30}{120} = 0,25 = 25 \, \%.$$

Interprétation : une commande sur quatre est passée à Casablanca.

💡 Variation relative (ou taux de variation)

Pour comparer une même grandeur entre deux périodes (ou deux situations), on calcule la **variation relative** :

$$\text{variation en } \% = \frac{\text{valeur finale} - \text{valeur initiale}}{\text{valeur initiale}} \times 100.$$

- Un résultat positif indique une hausse.
- Un résultat négatif indique une baisse.

Exemple – Variation relative

Le chiffre d'affaires d'un produit passe de 200 MAD à 240 MAD :

$$\frac{240 - 200}{200} \times 100 = 20 \, \%.$$

Interprétation : les ventes ont augmenté de 20 %.

Remarques

- Toujours préciser la période de référence (valeur initiale) et la période finale.
- Une variation de +100 % signifie un doublement ; une variation de -50 % signifie une division par deux.

Résumer une variable numérique

Lorsqu'on dispose d'une variable quantitative, on a besoin d'indicateurs qui résument sa **tendance centrale** (où se situent les valeurs) et sa **dispersion** (à quel point elles sont dispersées).

Définitions et formules

Indicateur	Définition / formule	Question à laquelle il répond
Moyenne ($\bar{x}$)	$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$	Quel serait le partage équitable du total ?

Indicateur	Définition / formule	Question à laquelle il répond
Médiane (Me)	Valeur qui partage la série ordonnée en deux groupes de même effectif	Quelle valeur est au milieu des observations ?
Minimum / Maximum	Plus petite et plus grande valeur observée	Quelles sont les extrêmes ?
Premier quartile (Q_1)	Médiane de la moitié inférieure de la série	Où se situent les 25 % les plus faibles ?
Troisième quartile (Q_3)	Médiane de la moitié supérieure de la série	Où se situent les 75 % les plus faibles ?
Écart interquartile (IQR)	$IQR = Q_3 - Q_1$	Quelle est la largeur de la moitié centrale des données ?
Écart-type (s)	$s = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2}$	Les valeurs sont-elles proches de la moyenne ?

💡 Calcul de la médiane et des quartiles

1. Médiane On note

$$x_1, x_2, \dots, x_n$$

les observations d'une variable quantitative et on note

$$x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(n)}$$

la série ordonnée.

— Si n est **impair** :

$$Me = x_{(\frac{n+1}{2})}$$

— Si n est **pair** :

$$Me = \frac{x_{(\frac{n}{2})} + x_{(\frac{n}{2}+1)}}{2}$$

2. Quartiles Q_1 et Q_3 (méthode de Tukey)

1. On calcule d'abord la **médiane** de toute la série.
2. On forme la **moitié inférieure** = toutes les observations **strictement inférieures** à la médiane.
3. On forme la **moitié supérieure** = toutes les observations **strictement supérieures** à la médiane.
4. Q_1 = médiane de la moitié inférieure.
5. Q_3 = médiane de la moitié supérieure.

❗ Pourquoi moyenne et médiane ensemble ?

- La moyenne est sensible aux valeurs extrêmes (outliers). Une seule commande très élevée peut la tirer fortement vers le haut.
- La médiane, elle, est robuste : elle ne change pas (ou peu) si l'on modifie les valeurs extrêmes.
- C'est pourquoi on présente presque toujours les deux indicateurs.

i Exemple 1

Montants de 9 commandes (en MAD) :

420, 95, 250, 110, 130, 160, 190, 80, 45

1. Série ordonnée

$$x_{(1)} = 45, x_{(2)} = 80, x_{(3)} = 95, x_{(4)} = 110, x_{(5)} = 130, x_{(6)} = 160, x_{(7)} = 190, x_{(8)} = 250, x_{(9)} = 420$$

2. Effectif : $n = 9$

3. Minimum et maximum

- min = 45 MAD

- max = 420 MAD

4. Moyenne

$$\bar{x} = \frac{45 + 80 + 95 + 110 + 130 + 160 + 190 + 250 + 420}{9} = \frac{1480}{9} \approx 164,44 \text{ MAD}$$

5. Médiane (n impair)

$$Me = x_{(\frac{9+1}{2})} = x_{(5)} = 130 \text{ MAD}$$

6. Quartiles

— Moitié inférieure (valeurs < 130) : 45, 80, 95, 110

$$\rightarrow Q_1 = \text{médiane de ces 4 valeurs} = \frac{80 + 95}{2} = 87,5 \text{ MAD}$$

— Moitié supérieure (valeurs > 130) : 160, 190, 250, 420

$$\rightarrow Q_3 = \text{médiane de ces 4 valeurs} = \frac{190 + 250}{2} = 220 \text{ MAD}$$

7. Écart interquartile

$$IQR = Q_3 - Q_1 = 220 - 87,5 = 132,5 \text{ MAD}$$

Interprétation

- La moyenne (164 MAD) est tirée vers le haut par la commande de 420 MAD.

- La médiane (130 MAD) est plus représentative du « client typique ».

- 50 % des commandes se situent entre 87,5 et 220 MAD (IQR = 132,5 MAD).

i Exemple 2

Montants de 10 commandes (en MAD) :

30, 55, 90, 70, 150, 125, 105, 180, 220, 380

1. Série ordonnée

$$x_{(1)} = 30, x_{(2)} = 55, x_{(3)} = 70, x_{(4)} = 90, x_{(5)} = 105, x_{(6)} = 125, x_{(7)} = 150, x_{(8)} = 180, x_{(9)} = 220, x_{(10)} = 380$$

2. Effectif : $n = 10$

3. Minimum et maximum

- min = 30 MAD

- max = 380 MAD

4. Moyenne

$$\bar{x} = \frac{30 + 55 + 70 + 90 + 105 + 125 + 150 + 180 + 220 + 380}{10} = 140,5 \text{ MAD}$$

5. Médiane (n pair)

$$Me = \frac{x_{(5)} + x_{(6)}}{2} = \frac{105 + 125}{2} = 115 \text{ MAD}$$

6. Quartiles

- Moitié inférieure (valeurs < 115) : 30, 55, 70, 90, 105
→ Q_1 = médiane de ces 5 valeurs = 70 MAD
- Moitié supérieure (valeurs > 115) : 125, 150, 180, 220, 380
→ Q_3 = médiane de ces 5 valeurs = 180 MAD

7. Écart interquartile

$$IQR = Q_3 - Q_1 = 180 - 70 = 110 \text{ MAD}$$

Interprétation

- La moyenne (140,5 MAD) est influencée par la commande de 380 MAD.
- La médiane (115 MAD) reflète mieux le centre de la distribution.
- La moitié centrale des commandes se situe entre 70 et 180 MAD ($IQR = 110 \text{ MAD}$).

💡 Calcul avec python (bibliothèque pandas)

```
resume = df["montant"].describe()
print(resume[["count", "mean", "50%", "min", "max", "25%", "75%"]])
```

La méthode `describe()` fournit :

- `count` : effectif
- `mean` : moyenne
- `50%` : médiane
- `min` / `max` : extrêmes
- `25%` / `75%` : quartiles Q_1 et Q_3

Elle ne remplace pas l'interprétation : un indicateur doit toujours être accompagné de son **unité**, de sa **période** et de la **population** étudiée.

⚠ Un indicateur n'est pas une conclusion

Une moyenne de 250 MAD ne dit pas si les commandes sont toutes proches de 250 MAD ou si quelques très grosses commandes compensent beaucoup de petites commandes.

Pour comprendre la forme de la distribution, on complète toujours la moyenne par la médiane, les quartiles (ou l'IQR) et, idéalement, un graphique (histogramme, boîte à moustaches...).

💡 Bonnes pratiques

- Toujours indiquer l'unité (MAD, kg, jours...).
- Préciser la période et le périmètre (tous les clients, un segment, une région...).
- Présenter moyenne **et** médiane lorsque la distribution est asymétrique.
- Utiliser l'IQR plutôt que l'écart-type si l'on suspecte des valeurs extrêmes.

3. Cycle de vie et mise en pratique avec Python

Le cycle de vie d'un projet d'analyse de données

Un projet d'analyse de données suit une succession d'étapes, rarement linéaire, que l'on peut résumer ainsi :

1. **Cadrage** : formuler clairement la question métier à laquelle l'analyse doit répondre.
2. **Collecte** : rassembler les données nécessaires (bases de données, fichiers, API, web scraping...).
3. **Nettoyage** : traiter les valeurs manquantes, les valeurs aberrantes, les doublons et les incohérences. C'est l'étape la plus chronophage en pratique.
4. **Transformation** : encoder, normaliser, recalculer certaines variables pour les rendre exploitables.
5. **Exploration (EDA)** : décrire statistiquement les données, en dégager des tendances, des corrélations, des anomalies.
6. **Visualisation et communication** : représenter graphiquement les résultats et les traduire en recommandations compréhensibles par un public non technique.

i Repère

Ce découpage est proche du cadre **OSEMN** (*Obtain, Scrub, Explore, Model, iNterpret*), popularisé par la communauté Data Science. Nous l'adaptions ici en retirant l'étape de modélisation prédictive (« Model »), hors périmètre de ce cours, et en insistant davantage sur la communication des résultats.

Comme pour tout cycle de projet, ces étapes sont **itératives** : une exploration révélant un problème de qualité de données ramène généralement à l'étape de nettoyage, et une visualisation peu convaincante amène souvent à revisiter l'exploration.

Écosystème d'outils Python pour l'analyse de données

Bibliothèques cœur

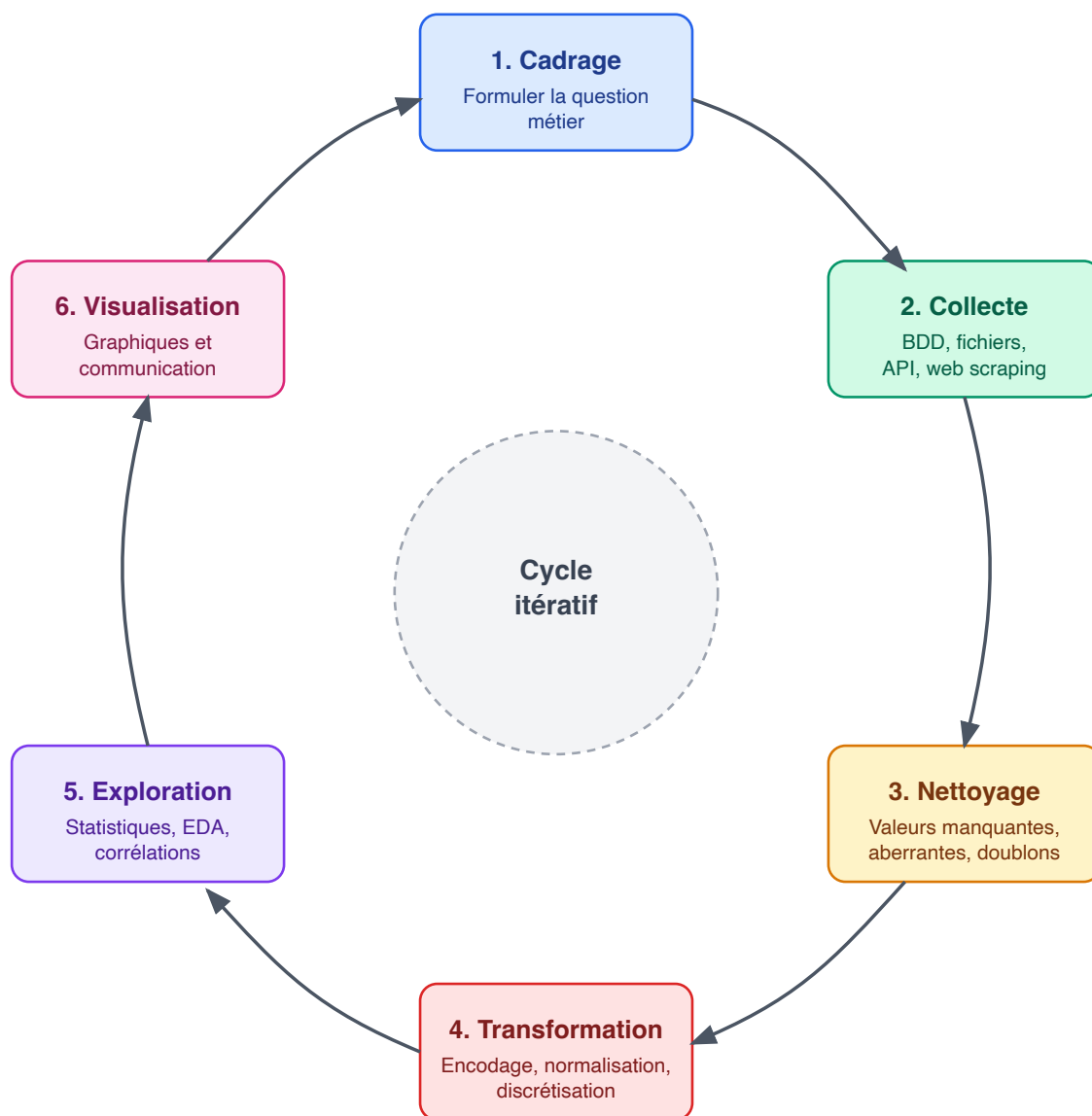
Comme pour le Machine Learning, Python s'impose comme langage principal pour l'analyse de données, grâce à la richesse de son écosystème scientifique et à la lisibilité de sa syntaxe.

TABLE 3 – Bibliothèques Python cœur de ce cours

Bibliothèque	Rôle
pandas	Manipulation de tableaux de données (DataFrame) : lecture, nettoyage, agrégation
numpy	Calcul numérique et vectoriel sous-jacent à pandas
matplotlib	Bibliothèque de visualisation de référence, hautement personnalisable
seaborn	Visualisation statistique construite sur matplotlib , orientée exploration rapide

Environnements de travail

Comme dans le reste de la formation, nous privilégions les **notebooks Jupyter** (JupyterLab, Google Colab) pour l'exploration interactive et la visualisation immédiate cellule par cellule, et **Visual Studio Code** si l'étudiant voulait utiliser des fichiers python ou ouvrir jupyter notebook dans cet environnement.



Une exploration ou une visualisation décevante ramène souvent vers les étapes 2, 3 ou 4.

FIGURE 2 – Le cycle de vie d'un projet d'analyse de données

Gestion de projet avec uv

Un projet de *Machine Learning* ne dépend pas uniquement du code que nous écrivons. Il repose également sur un **environnement logiciel** : une version de Python, un ensemble de bibliothèques, des versions compatibles de ces bibliothèques et différents outils nécessaires à l'exécution du projet.

La gestion de cet environnement est importante dès qu'un projet doit être **reproduit, partagé, maintenu ou déployé**. Deux utilisateurs disposant du même code peuvent, par exemple, rencontrer des problèmes différents si leurs versions de Python ou de certaines dépendances ne sont pas les mêmes. La gestion des dépendances constitue ainsi une composante essentielle de la reproductibilité et de la maintenance d'un projet.

Dans ce cours, nous présenterons d'abord l'approche classique de l'écosystème Python, puis nous adopterons uv comme outil de gestion de nos projets.

L'approche classique : Python, venv et pip

L'écosystème Python fournit déjà les outils nécessaires pour créer un environnement isolé et installer les bibliothèques d'un projet. Le module **venv** est l'outil standard de Python pour créer des environnements virtuels. Ceux-ci permettent d'isoler les dépendances d'un projet de celles installées dans les autres projets ou dans l'installation générale de Python.

L'installation des bibliothèques s'effectue généralement avec **pip**, l'installateur de paquets Python. Les bibliothèques sont notamment disponibles sur **PyPI**, le Python Package Index.

Une configuration classique peut donc être représentée de manière simplifiée par :

```
python -m venv .venv
```

puis, dans l'environnement virtuel :

```
python -m pip install numpy pandas scikit-learn
```

Les dépendances peuvent ensuite être consignées dans un fichier **requirements.txt**, par exemple :

```
numpy  
pandas  
scikit-learn
```

Cette approche est parfaitement valable et reste largement utilisée. Le [Python Packaging User Guide](#) présente d'ailleurs cette organisation ainsi que les pratiques modernes de gestion et de distribution des projets Python.

Elle présente cependant une certaine fragmentation : la création de l'environnement, la gestion des dépendances et leur installation sont assurées par des mécanismes distincts. Pour des projets plus structurés, il devient également utile de disposer d'une déclaration centralisée des dépendances et d'un mécanisme de verrouillage permettant de conserver une résolution précise de l'environnement.

Vers une gestion moderne du projet avec pyproject.toml

Les projets Python modernes utilisent notamment le fichier **pyproject.toml**. Ce fichier permet de centraliser différentes informations relatives au projet, notamment ses métadonnées et ses dépendances.

Il constitue aujourd'hui un élément central de l'écosystème de *packaging* Python et est utilisé par de nombreux outils modernes.

Dans ce cours, nous allons nous appuyer sur **uv**, développé par Astral, pour gérer nos projets Python.

Pourquoi uv ?

`uv` est un gestionnaire de projets et de paquets Python qui regroupe dans un même outil plusieurs opérations auparavant réalisées avec différents outils. Il permet notamment de gérer les projets Python, les environnements virtuels, les versions de Python, les dépendances et leur résolution, ainsi que l'exécution des commandes dans l'environnement du projet.

La documentation officielle de [uv](#) présente notamment les fonctions suivantes :

- création d'un projet avec `uv init` ;
- ajout et suppression de dépendances ;
- gestion de l'environnement virtuel du projet ;
- synchronisation des dépendances ;
- création et mise à jour du fichier de verrouillage ;
- exécution de commandes dans l'environnement du projet ;
- gestion de projets plus complexes au moyen de *workspaces*.

L'intérêt de `uv` dans notre cours est donc moins de remplacer une commande particulière que de fournir une **organisation cohérente de l'ensemble du projet Python**.

`pyproject.toml`, `uv.lock` et `.venv`

Avec `uv`, trois éléments sont particulièrement importants.

Le fichier `pyproject.toml` décrit le projet et ses besoins, notamment les dépendances et la version de Python requise.

Le fichier `uv.lock` enregistre la résolution précise des dépendances du projet. Contrairement au fichier `pyproject.toml`, qui exprime les besoins du projet de manière générale, le fichier de verrouillage conserve les versions effectivement résolues. Il peut ainsi être versionné avec le projet afin de faciliter la reconstruction d'un environnement cohérent sur différentes machines. La documentation officielle de [uv](#) décrit précisément le rôle de `uv.lock`.

Enfin, `uv` utilise un environnement virtuel associé au projet, généralement situé dans `.venv`.

On peut donc représenter schématiquement l'organisation d'un projet ainsi :

```
projet/  
  .venv/  
  pyproject.toml  
  uv.lock  
  ...
```

Le principe peut être résumé ainsi :

`pyproject.toml` décrit les besoins du projet ; `uv.lock` conserve leur résolution précise ; `.venv` contient l'environnement dans lequel le projet est exécuté.

La synchronisation entre ces éléments est prise en charge par `uv`. La documentation officielle explique notamment que les opérations telles que [uv sync](#) et [uv run](#) peuvent mettre à jour l'environnement à partir du fichier de verrouillage.

Pourquoi uv plutôt qu'Anaconda ou Conda ?

Il existe d'autres solutions parfaitement valables pour gérer les environnements Python. L'une des plus connues est l'écosystème [Conda](#), qui permet notamment de créer, gérer, partager et supprimer des environnements contenant différentes versions de Python et de paquets.

Notre choix de uv dans ce cours ne signifie donc pas que Conda ou [Anaconda](#) sont inadaptés. Il s'agit d'un **choix pédagogique et technique** correspondant à l'organisation que nous souhaitons adopter.

Nous retenons uv principalement pour les raisons suivantes :

1. **Une approche directement centrée sur les projets Python.** uv s'intègre naturellement aux mécanismes modernes du *packaging* Python, notamment `pyproject.toml` et l'écosystème PyPI.
2. **Une gestion unifiée du projet.** Un même outil permet de gérer l'environnement, les dépendances, leur résolution, le verrouillage et l'exécution du projet.
3. **Une structure explicite et versionnable.** L'association de `pyproject.toml` et `uv.lock` fournit une description claire du projet et de son environnement de dépendances.
4. **Une bonne adéquation avec les projets du cours.** Les projets étudiés utiliseront principalement des bibliothèques de l'écosystème Python, telles que `numpy`, `pandas`, `matplotlib` et `scikit-learn`, ainsi que les outils Jupyter. uv fournit une organisation cohérente pour leur gestion dans un projet Python local.
5. **Une continuité entre expérimentation et développement.** Les étudiants pourront commencer par explorer les données et les modèles dans des notebooks, puis retrouver le même environnement de projet lorsqu'ils passeront à une organisation plus structurée du code Python.

Le choix de uv répond donc avant tout aux besoins de ce cours : disposer d'un environnement Python **isolé, explicite, reproductible et facile à partager**, tout en introduisant progressivement les pratiques modernes de gestion de projet.

Tendances récentes de l'écosystème

! Zoom sur les tendances actuelles

L'écosystème de l'analyse de données évolue rapidement. Sans remplacer les outils historiques présentés ci-dessus, plusieurs bibliothèques récentes gagnent en popularité et méritent d'être connues :

- **Polars** : une alternative à `pandas`, écrite en Rust, offrant des performances nettement supérieures sur de gros volumes de données grâce à l'exécution parallèle et à l'évaluation paresseuse (*lazy evaluation*).
- **DuckDB** : un moteur de base de données analytique embarqué, permettant d'exécuter des requêtes SQL directement sur des fichiers CSV ou Parquet, sans serveur, avec des performances proches d'un entrepôt de données.
- **Outils de profilage automatique** (`ydata-profiling`, `Sweetviz`) : génèrent en une commande un rapport exploratoire complet d'un jeu de données (statistiques, distributions, corrélations, valeurs manquantes).¹
- **Assistants d'analyse par IA générative** : des outils permettant d'interroger un `DataFrame` en langage naturel commencent à s'intégrer aux environnements de data analysis, sans se substituer à une compréhension rigoureuse des données par l'analyste (l'humain !!).

Ces outils ne sont pas obligatoires dans ce cours, mais seront mentionnés ponctuellement à titre de complément, car ils représentent une évolution significative des pratiques professionnelles actuelles.

Les métiers de la donnée

1. voir le chapitre sur l'exploration des données.

TABLE 4 – Les principaux métiers autour de la donnée

Métier	Mission principale	Compétences clés
Data Analyst	Explorer, visualiser et interpréter les données pour éclairer une décision	SQL, pandas , visualisation, statistiques descriptives
BI Analyst / BI Developer	Construire et maintenir des tableaux de bord et indicateurs récurrents	SQL, outils de BI (Power BI, Tableau), modélisation de données
Data Scientist	Concevoir des modèles statistiques et prédictifs	Statistique inférentielle, Machine Learning, programmation
Data Engineer	Construire et fiabiliser les pipelines d'acquisition et de stockage des données	Bases de données, orchestration, ingénierie logicielle

Ce cours prépare avant tout au cœur du métier de **Data Analyst**, tout en posant des bases indispensables aux trois autres profils.

Exercices

Exercice 0.0.1 (Vrai ou Faux).

- La Business Intelligence répond principalement à la question « pourquoi ? ».
- Une analyse diagnostique s'appuie généralement sur une analyse descriptive préalable.
- Le Machine Learning est un prérequis indispensable pour produire une analyse descriptive.

Exercice 0.0.2 (Classer les questions). Pour chacune des questions suivantes, indiquez le type d'analyse correspondant (descriptive, diagnostique, prédictive ou prescriptive) :

- Combien de visiteurs le site web a-t-il reçus la semaine dernière ?
- Pourquoi le taux d'abandon de panier a-t-il augmenté en juillet ?
- Quelle campagne publicitaire faut-il privilégier pour la rentrée ?

Exercice 0.0.3 (Étude de cas courte). Une association marocaine de protection de l'environnement souhaite comprendre l'évolution du volume de déchets collectés dans trois villes sur les deux dernières années, puis identifier les facteurs expliquant les écarts entre villes.

- Quelles étapes du cycle de vie présenté en Figure 2 cette association devra-t-elle suivre ?
- À quel(s) type(s) d'analyse (Table 2) ce projet correspond-il ?

Exercice 0.0.4 (Moyenne, médiane et fréquence). Les montants de cinq commandes sont : 80, 100, 100, 120 et 600 MAD.

- Calculez la moyenne et la médiane.
- Quel indicateur décrit le mieux une commande « typique » ? Pourquoi ?
- Quel est l'effectif et la fréquence des commandes d'au moins 100 MAD ?

Exercice 0.0.5 (Lire une variation). Le nombre de visiteurs d'un site passe de 2 000 en avril à 2 400 en mai, puis à 2 280 en juin.

- Calculez la variation en pourcentage entre avril et mai.

- b. Calculez la variation en pourcentage entre mai et juin.
- c. La baisse de mai à juin annule-t-elle entièrement la hausse précédente ?

Exercice 0.0.6 (Identifier les indicateurs utiles). Pour chacune des questions, choisissez l'indicateur le plus directement utile : effectif, fréquence, moyenne, médiane, minimum/maximum ou IQR.

- a. Quelle part des clients utilise le paiement par carte ?
- b. Quel est le montant central des salaires ?
- c. Quelle est la largeur de la moitié centrale des durées de livraison ?
- d. Combien de commandes ont été enregistrées ?

Exercice 0.0.7 (Reconnaître le type de variable). Classez les variables suivantes en qualitative, quantitative ou temporelle : `quartier`, `nombre_de_produits`, `date_commande`, `satisfaction` codée « faible / moyenne / élevée ».

Exercice 0.0.8 (Choisir une analyse). Une responsable constate que les ventes sont plus faibles en août. Elle veut d'abord mesurer la baisse, puis comprendre si elle concerne seulement une région.

- a. Quel type d'analyse répond à la première question ?
- b. Quelle comparaison simple peut aider pour la seconde ?
- c. Pourquoi faut-il éviter de conclure à une cause avant d'avoir comparé les données ?

Préparation des données

💡 Objectifs du chapitre

À la fin de ce chapitre, l'étudiant sera capable de :

- **Identifier** les principales sources de collecte de données (bases de données, fichiers plats, API, web scraping) ;
- **Diagnostiquer** les problèmes de qualité d'un jeu de données (valeurs manquantes, aberrantes, dupliquées) ;
- **Appliquer** des stratégies adaptées de traitement des valeurs manquantes et aberrantes avec **pandas** ;
- **Transformer** des variables (encodage, normalisation, discrétisation) en vue de leur exploration ou de leur visualisation.

Si les données sont de mauvaise qualité, corrompues ou biaisées, aucune analyse — aussi soignée soit-elle — ne pourra produire de conclusions fiables (*Garbage In, Garbage Out*).

- Principe fondamental de l'analyse de données

Avant de pouvoir explorer ou visualiser un jeu de données, il faut d'abord le **rassembler** puis le **rendre exploitable**. Ce chapitre couvre les étapes 2, 3 et 4 du cycle de vie présenté au chapitre précédent (Figure 2) : où trouver les données, comment détecter et corriger leurs défauts, puis comment les transformer pour l'exploration et la visualisation.

i Réflexe de débutant : connaître son tableau

Avant toute modification, conservez une copie du fichier original et rédigez un petit **dictionnaire de données**. Pour chaque colonne, notez son nom, sa signification, son unité, son type attendu et les valeurs possibles. Par exemple, **prix** peut être un nombre positif en MAD, tandis que **type_bien** peut prendre les valeurs « appartement », « maison » ou « terrain ». Ce dictionnaire évite de corriger une valeur qui était en réalité valide.

1. Collecte de données

1.1. Les bases de données

La source la plus courante en entreprise reste la base de données interne :

- **relationnelle (SQL)** : données organisées en tables reliées par des clés (PostgreSQL, MySQL, SQL Server), interrogées avec le langage **SQL** ;
- **non-relationnelle (NoSQL)** : données stockées sous une forme plus flexible — documents JSON (MongoDB), paires clé-valeur (Redis) — adaptées à des données peu structurées ou à très grande échelle.

pandas permet de charger directement le résultat d'une requête SQL dans un **DataFrame** :

```
import pandas as pd
import sqlite3

connexion = sqlite3.connect("ventes.db")
df = pd.read_sql("SELECT * FROM transactions WHERE annee = 2025", connexion)
```

1.2. Les fichiers plats et formats de données

En dehors des bases de données, les données sont fréquemment échangées sous forme de fichiers :

TABLE 1 – Principaux formats de données rencontrés en analyse de données

Format	Caractéristiques	Lecture avec <code>pandas</code>
CSV	Texte tabulaire, très répandu, lisible par un humain	<code>pd.read_csv()</code>
JSON	Structure hiérarchique (objets imbriqués), courant pour les API	<code>pd.read_json()</code>
Excel (.xlsx)	Tableur, souvent utilisé en entreprise	<code>pd.read_excel()</code>
Parquet	Format binaire colonnaire, compressé, optimisé pour l'analytique	<code>pd.read_parquet()</code>

! Zoom sur les tendances : le format Parquet

Contrairement au CSV, le format **Parquet** stocke les données par colonne plutôt que par ligne. Cela permet de ne lire que les colonnes réellement nécessaires à une analyse, de bénéficier d'une compression efficace et de conserver les types de données (un entier reste un entier, pas une chaîne de caractères à réinterpréter). Il est aujourd'hui privilégié pour les jeux de données volumineux, et s'utilise nativement avec `pandas`, `Polars` ou `DuckDB`.

1.3. Les API

Une **API** (*Application Programming Interface*) permet d'obtenir des données structurées de façon normalisée, sans avoir à analyser du HTML :

- les **API REST**, les plus répandues, où chaque ressource est accessible via une URL et une méthode HTTP (GET, POST...), avec une réponse généralement au format JSON ;
- les **API GraphQL**, plus récentes, qui permettent au client de préciser exactement les champs souhaités en une seule requête.

```
import requests
import pandas as pd

reponse = requests.get("https://api.exemple.org/ventes?annee=2025")
df = pd.json_normalize(reponse.json()["resultats"])
```

1.4. Le web scraping

Lorsque les données ne sont disponibles que sur des pages web, on recourt au **web scraping** : l'extraction automatisée d'informations à partir du code HTML. Les bibliothèques Python de référence sont **BeautifulSoup** (analyse d'une page unique) et **Scrapy** (collecte à grande échelle, avec gestion de la pagination et des files d'attente).

⚠ Éthique et légalité du web scraping

Avant de collecter des données sur un site, il convient de vérifier son fichier `robots.txt`, ses conditions générales d'utilisation, ainsi que le cadre réglementaire applicable à la protection des données personnelles (RGPD, ou la loi marocaine 09-08), en particulier si des données à caractère personnel sont concernées.

1.5. Open data et capteurs / IoT

Deux autres sources méritent d'être mentionnées :

- les **portails d'open data** (données publiques mises à disposition par des institutions, ex. données démographiques, économiques ou climatiques) ;
- les **capteurs et objets connectés (IoT)**, qui génèrent des flux de données en continu (température, géolocalisation, fréquentation), souvent collectés via des files de messages (ex. MQTT, Kafka) avant d'être stockés pour l'analyse.

2. Nettoyage des données

Une fois collectées, les données contiennent presque toujours des imperfections. Le nettoyage est l'étape la plus chronophage d'un projet d'analyse — souvent estimée à 60-80 % du temps réel passé sur un projet.

Un nettoyage fiable vérifie au moins cinq dimensions :

1. **complétude** : les informations importantes sont-elles présentes ?
2. **validité** : les valeurs respectent-elles les règles attendues (un prix positif, une date valide) ?
3. **unicité** : une même observation n'est-elle pas enregistrée deux fois ?
4. **cohérence** : les colonnes et les unités sont-elles compatibles entre elles ?
5. **actualité** : les dates couvrent-elles la période utile à la question ?

Il faut garder une trace de chaque correction : colonne concernée, règle appliquée, nombre de lignes modifiées et justification. Cette documentation permet de revenir en arrière et d'expliquer les résultats.

2.1. Les valeurs manquantes

Une valeur manquante peut avoir des origines très différentes, ce qui conditionne la stratégie à adopter :

TABLE 2 – Typologie des valeurs manquantes

Type	Signification	Exemple
MCAR (<i>Missing Completely At Random</i>)	L'absence ne dépend d'aucune variable, ni observée ni cachée	Un capteur tombe en panne aléatoirement
MAR (<i>Missing At Random</i>)	L'absence dépend d'une autre variable observée	Les hommes répondent moins souvent à une question sur le revenu

Type	Signification	Exemple
MNAR (<i>Missing Not At Random</i>)	L'absence dépend de la valeur manquante elle-même	Les personnes aux revenus très élevés refusent de répondre

Ces trois étiquettes servent surtout à réfléchir à la cause d'une absence. Il n'est pas demandé de réaliser un test statistique pour les distinguer dans ce cours : l'étudiant doit surtout éviter d'imputer automatiquement sans comprendre ce que signifie la valeur manquante.

Avec **pandas**, on détecte et on traite les valeurs manquantes ainsi :

```
# Détection
df.isna().sum()                # nombre de valeurs manquantes par colonne
df.isna().mean().round(2) * 100 # pourcentage de valeurs manquantes

# Suppression
df.dropna(subset=["prix"])      # supprime les lignes sans prix

# Imputation
df["surface"] = df["surface"].fillna(df["surface"].median())
df["quartier"] = df["quartier"].fillna("Non renseigné")
```

💡 Comment choisir entre suppression et imputation ?

- Si la proportion de valeurs manquantes est **faible** (quelques pourcents) et que le mécanisme semble MCAR, la **suppression** des lignes concernées est souvent acceptable.
- Si la proportion est **élevée**, ou si l'absence porte sur une variable centrale de l'analyse, on préfère une **imputation** (moyenne, médiane, mode, ou une valeur explicite comme « Non renseigné »), en gardant à l'esprit qu'elle introduit un biais qu'il faut documenter.

2.2. Les valeurs aberrantes (*outliers*)

Une valeur aberrante est une observation qui s'écarte fortement du reste de la distribution. Deux méthodes usuelles permettent de les détecter :

- la méthode du **score Z**, qui signale une observation aberrante lorsque $|z| = \left| \frac{x - \bar{x}}{\sigma} \right| > 3$;
- la méthode de l'**écart interquartile (IQR)**, plus robuste aux distributions asymétriques : on considère comme aberrante toute valeur en dehors de l'intervalle $[Q_1 - 1,5 \times IQR, Q_3 + 1,5 \times IQR]$, avec $IQR = Q_3 - Q_1$.

```
q1, q3 = df["prix"].quantile([0.25, 0.75])
iqr = q3 - q1
borne_basse, borne_haute = q1 - 1.5 * iqr, q3 + 1.5 * iqr

aberrants = df[(df["prix"] < borne_basse) | (df["prix"] > borne_haute)]
```

La **boîte à moustaches** (*boxplot*), présentée au chapitre suivant sur la visualisation, est l'outil graphique privilégié pour repérer visuellement ces valeurs.

⚠ Une valeur aberrante n'est pas toujours une erreur

Avant de supprimer une valeur aberrante, il faut se demander si elle correspond à une **erreur de saisie** (à corriger ou supprimer) ou à un **phénomène réel mais rare** (par exemple une vente exceptionnelle), qu'il peut être pertinent de conserver, voire d'étudier spécifiquement.

2.3. Doublons et incohérences

```
df.duplicated().sum()          # nombre de lignes strictement dupliquées
df = df.drop_duplicates()

df["ville"] = df["ville"].str.strip().str.title()  # espaces superflus, casse
df["date"] = pd.to_datetime(df["date"], errors="coerce")  # type de données cohérent
```

Les incohérences les plus fréquentes concernent la **casse** et les **espaces superflus** dans les chaînes de caractères (« casablanca » vs « Casablanca »), les **types de données** mal interprétés (une date lue comme texte), et les **unités hétérogènes** (des prix exprimés tantôt en MAD, tantôt en euros).

2.4. Vers une démarche de qualité des données

! Zoom sur les tendances : la validation systématique de la qualité

Sur des projets d'analyse récurrents (tableaux de bord alimentés quotidiennement, par exemple), il devient risqué de vérifier la qualité des données « à l'œil » à chaque exécution. Des bibliothèques comme **Great Expectations** permettent de définir des règles de validation explicites (« la colonne **prix** doit toujours être positive », « la colonne **date** ne doit jamais être manquante ») et de les exécuter automatiquement à chaque nouvelle collecte de données, afin de détecter une dérive de qualité avant qu'elle n'affecte l'analyse.

3. Transformation des données

Une fois nettoyées, les données doivent souvent être reformatées avant l'exploration ou la visualisation.

3.1. Encodage des variables catégorielles

De nombreux outils statistiques et graphiques manipulent plus facilement des valeurs numériques que du texte. On distingue deux stratégies principales :

- l'**encodage ordinal**, adapté aux catégories qui possèdent un ordre naturel (ex. « Faible » < « Moyen » < « Élevé ») ;
- l'**encodage one-hot** (variables indicatrices), adapté aux catégories sans ordre (ex. les quartiers d'une ville), qui crée une colonne binaire par catégorie.

```
# Encodage ordinal
ordre = {"Faible": 0, "Moyen": 1, "Élevé": 2}
df["niveau_encode"] = df["niveau"].map(ordre)

# Encodage one-hot
df_encode = pd.get_dummies(df, columns=["quartier"])
```

Note

L'encodage détaillé des variables (avec `scikit-learn`, gestion des catégories rares, etc.) sera approfondi dans le module de Machine Learning. Dans ce cours, il est surtout utile pour faciliter certains graphiques et calculs de corrélation présentés au chapitre suivant.

3.2. Normalisation et standardisation

Lorsque plusieurs variables numériques ont des échelles très différentes (ex. une surface en m^2 et un prix en millions de MAD), il est parfois utile de les ramener à une échelle comparable, notamment pour certaines visualisations combinées :

- la **normalisation min-max** ramène chaque valeur dans l'intervalle $[0, 1]$: $x' = \frac{x - x_{min}}{x_{max} - x_{min}}$;
- la **standardisation** (score Z) centre la variable autour de 0 avec un écart-type de 1 : $x' = \frac{x - \bar{x}}{\sigma}$.

```
df["prix_norm"] = (df["prix"] - df["prix"].min()) / (df["prix"].max() - df["prix"].min())
df["prix_standard"] = (df["prix"] - df["prix"].mean()) / df["prix"].std()
```

3.3. Discrétisation (*binning*)

La discrétisation consiste à transformer une variable numérique continue en catégories, souvent utile pour simplifier une visualisation (par exemple des tranches d'âge plutôt qu'un âge exact) :

```
df["tranche_age"] = pd.cut(
    df["age"],
    bins=[0, 18, 35, 60, 120],
    labels=["Mineur", "Jeune adulte", "Adulte", "Senior"]
)
```

3.4. Quelques transformations usuelles supplémentaires

- **Extraction de composantes temporelles** : à partir d'une colonne `date`, on extrait fréquemment l'année, le mois, le jour de semaine (`df["date"].dt.month`), utiles pour l'exploration de séries temporelles.
- **Agrégats dérivés** : création de nouvelles colonnes résumant une information (ex. le prix au mètre carré = `prix / surface`), qui facilitent grandement l'exploration et la comparaison.

Ne pas transformer sans objectif

Une transformation n'est pas automatiquement une amélioration. Par exemple, remplacer toutes les valeurs manquantes par zéro change la signification de la variable : « absence d'information » devient « valeur nulle ». Avant chaque transformation, demandez-vous quelle question elle facilite et si son effet sera documenté.

Exercices

Exercice 0.0.1 (Choisir une stratégie de traitement des valeurs manquantes). Un jeu de données de candidatures à un emploi comporte une colonne `salaire_souhaite` manquante pour 40 % des lignes. Une analyse rapide montre que les candidats les plus expérimentés (souvent plus exigeants sur le salaire) sont surreprésentés parmi les valeurs manquantes.

- À quel type de valeur manquante (Table 2) correspond cette situation ?
- Recommanderiez-vous une suppression ou une imputation ? Justifiez.

Exercice 0.0.2 (Détection d'outliers). Pour une variable `prix` avec $Q_1 = 200\,000$ et $Q_3 = 450\,000$ (en MAD), calculez les bornes de détection des valeurs aberrantes selon la méthode de l'IQR.

Exercice 0.0.3 (Encodage). Pour chacune des variables suivantes, indiquez si un encodage **ordinal** ou **one-hot** est le plus adapté, et pourquoi :

- Le niveau d'étude (« Bac », « Bac+2 », « Bac+3 », « Bac+5 ») ;
- La couleur préférée d'un client (« Rouge », « Bleu », « Vert »).

Exercice 0.0.4 (Diagnostic de qualité). Un fichier de ventes contient 1 000 lignes. La colonne `date` comporte 20 valeurs manquantes, `prix` contient 5 valeurs négatives et 12 lignes sont des doublons exacts.

- Quel pourcentage de dates est manquant ?
- Quel problème doit être vérifié en priorité avant de calculer le chiffre d'affaires ?
- Les doublons doivent-ils être supprimés automatiquement ? Justifiez.

Exercice 0.0.5 (Choisir une stratégie d'imputation). Associez chaque situation à une stratégie raisonnable : supprimer quelques lignes, utiliser la médiane, créer la catégorie « Non renseigné » ou ne pas imputer sans investigation.

- 1 % des lignes n'ont pas de surface, et la surface est numérique.
- 35 % des lignes n'ont pas de quartier.
- Une date manquante empêche de savoir à quelle période appartient une vente.
- Une variable `remise` a 2 % de valeurs manquantes dans une distribution très asymétrique.

Exercice 0.0.6 (Calculer une normalisation min-max). Une variable `score` varie de 20 à 80. Calculez la valeur normalisée d'un score égal à 50 avec :

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}.$$

Que signifie le résultat ?

Exercice 0.0.7 (Unités et cohérence). Une colonne `distance` contient les valeurs 5, 12 et 8 pour trois trajets, mais le dictionnaire indique que certaines lignes sont en kilomètres et d'autres en mètres.

- Pourquoi est-il dangereux de calculer une moyenne directement ?
- Quelle action doit être réalisée avant l'analyse ?
- Est-ce un problème de valeur manquante, de validité ou de cohérence ?

Exercice 0.0.8 (Discrétisation et interprétation). On regroupe les âges selon les intervalles $[0,18[$, $[18,35[$, $[35,60[$ et $[60,120]$.

- Dans quelle classe place-t-on les âges 18, 34, 35 et 60 ?
- Quel avantage apporte cette transformation pour un tableau de bord ?
- Quel risque faut-il signaler ?

Exploration des données

💡 Objectifs du chapitre

À la fin de ce chapitre, l'étudiant sera capable de :

- **Calculer et interpréter** les principales statistiques descriptives (tendance centrale, dispersion, forme) ;
- **Distinguer** l'analyse univariée, bivariée et multivariée ;
- **Mesurer** une corrélation ou une association entre deux variables et en connaître les limites ;
- **Construire** des tableaux d'agrégation (**groupby**, tableaux croisés dynamiques) pour résumer un jeu de données ;
- **Utiliser** un outil de profilage automatique pour accélérer une première exploration.

L'**analyse exploratoire de données** (*Exploratory Data Analysis*, EDA), popularisée par le statisticien John Tukey, consiste à examiner un jeu de données déjà nettoyé afin d'en dégager les tendances principales, les relations entre variables et les éventuelles anomalies restantes — avant toute conclusion définitive ou toute visualisation destinée à un public externe.

1. Statistiques descriptives

1.1. Tendance centrale

TABLE 1 – Indicateurs de tendance centrale

Indicateur	Définition	Sensibilité aux valeurs extrêmes
Moyenne	Somme des valeurs divisée par leur nombre	Élevée
Médiane	Valeur qui sépare la distribution en deux moitiés égales	Faible
Mode	Valeur la plus fréquente	Faible (mais peu informatif pour une variable continue)

```
df["prix"].mean()
df["prix"].median()
df["quartier"].mode()[0]
```

⚠️ Moyenne ou médiane ?

Sur une distribution fortement asymétrique — typiquement les salaires ou les prix de l'immobilier — la moyenne peut être fortement tirée vers le haut par quelques valeurs extrêmes. La **médiane** donne alors une image plus représentative de la « valeur typique ».

1.2. Dispersion

- l'**écart-type** (σ) et la **variance** (σ^2) mesurent l'écart moyen à la moyenne ;
- l'**écart interquartile** ($IQR = Q_3 - Q_1$), déjà rencontré au chapitre précédent, mesure la dispersion de la moitié centrale des données, de façon robuste aux valeurs extrêmes ;
- l'**étendue** (maximum – minimum) donne une idée rapide, mais très sensible aux valeurs aberrantes.

```
df["prix"].std()
df["prix"].var()
df["prix"].quantile([0.25, 0.5, 0.75])
```

Pour un petit tableau, on peut calculer les indicateurs à la main. Pour les valeurs 2, 4, 4, 6, 9, la moyenne est $25/5 = 5$ et la médiane est 4. L'étendue vaut $9 - 2 = 7$. Les quartiles dépendent de la convention de calcul utilisée par l'outil ; dans un cours débutant, l'important est de préciser la convention et de comparer les résultats avec la même méthode.

L'écart-type est exprimé dans la même unité que la variable, contrairement à la variance. Il est utile pour décrire la dispersion, mais il est sensible aux valeurs extrêmes. L'IQR est souvent plus facile à interpréter pour une distribution asymétrique. Le **coefficient de variation**, défini par $\sigma/\bar{x}$ lorsque la moyenne est positive et non nulle, permet parfois de comparer la dispersion relative de variables ayant des échelles différentes ; il est présenté ici comme un repère et non comme un indicateur obligatoire.

i Priorité pour débiter

Les indicateurs indispensables sont l'effectif, la moyenne, la médiane, le minimum, le maximum, les quartiles et l'écart-type ou l'IQR. L'asymétrie, l'aplatissement et le coefficient de variation sont des compléments : il suffit d'en comprendre l'idée et de savoir quand demander une explication supplémentaire.

1.3. Forme de la distribution

Deux indicateurs complètent la description d'une distribution :

- l'**asymétrie** (*skewness*) : une valeur positive indique une distribution étalée vers la droite (queue de valeurs élevées), une valeur négative vers la gauche ;
- l'**aplatissement** (*kurtosis*) : indique si la distribution comporte davantage de valeurs extrêmes qu'une distribution normale.

```
df["prix"].skew()
df["prix"].kurt()
```

La méthode `describe()` de `pandas` regroupe l'essentiel de ces indicateurs en une seule commande :

```
df.describe()
```

2. Analyse univariée, bivariable et multivariée

TABLE 2 – Les trois niveaux de l’analyse exploratoire

Niveau d’analyse	Nombre de variables étudiées simultanément	Exemple
Univariée	Une seule	Distribution des prix de vente
Bivariée	Deux	Relation entre surface et prix
Multivariée	Trois ou plus	Relation entre surface, quartier et prix

L’**analyse univariée** décrit une variable isolément (statistiques descriptives, distribution). L’**analyse bivariée** cherche une relation entre deux variables — par exemple à l’aide d’un tableau croisé pour deux variables catégorielles, ou d’un coefficient de corrélation pour deux variables numériques. L’**analyse multivariée** étend cette logique à plusieurs variables simultanément, souvent à l’aide de tableaux d’agrégation ou de visualisations combinant plusieurs dimensions (couleur, taille, facettes), présentées au chapitre suivant.

3. Corrélations et associations

3.1. Variables numériques : Pearson et Spearman

- le **coefficient de corrélation de Pearson** (r , entre -1 et 1) mesure l’intensité d’une relation **linéaire** entre deux variables numériques ;
- le **coefficient de corrélation de Spearman** mesure une relation **monotone** (pas nécessairement linéaire), en travaillant sur les rangs des valeurs plutôt que sur les valeurs elles-mêmes — plus robuste aux valeurs aberrantes.

```
df[["surface", "prix"]].corr(method="pearson")
df[["surface", "prix"]].corr(method="spearman")
```

⚠ Corrélation n’est pas causalité

Une corrélation élevée entre deux variables ne signifie pas que l’une provoque l’autre : elle peut résulter d’une **variable cachée** commune, ou d’une pure coïncidence statistique. Un exemple classique est la corrélation observée entre les ventes de glaces et le nombre de noyades : les deux augmentent en été, sans lien de cause à effet direct entre elles.

3.2. Variables catégorielles : tableaux croisés et V de Cramér

Pour deux variables catégorielles, on utilise un **tableau croisé** (*crosstab*), éventuellement complété par le **V de Cramér**, qui résume en un seul indicateur (entre 0 et 1) l’intensité de l’association :

```
tableau_croise = pd.crosstab(df["quartier"], df["type_bien"])
```

4. Agrégation : *groupby* et tableaux croisés dynamiques

L'outil central de l'exploration multivariée avec **pandas** est la méthode `groupby()`, qui permet de calculer un indicateur pour chaque sous-groupe d'une variable catégorielle :

```
df.groupby("quartier")["prix"].mean().sort_values(ascending=False)

df.groupby(["quartier", "type_bien"]).agg(
    prix_moyen=("prix", "mean"),
    nombre_biens=("prix", "count")
)
```

Le **tableau croisé dynamique** (*pivot table*) offre une présentation plus lisible lorsque l'on croise deux dimensions catégorielles avec une mesure numérique :

```
pd.pivot_table(
    df,
    values="prix",
    index="quartier",
    columns="type_bien",
    aggfunc="mean"
)
```

Ces deux outils constituent la base de la quasi-totalité des tableaux de bord et des rapports produits par un Data Analyst.

5. Choisir une visualisation simple

Les statistiques répondent à des questions numériques ; les graphiques rendent les formes et les comparaisons plus faciles à voir. Pour débiter, on peut retenir les associations suivantes :

Question	Graphique conseillé	Ce que l'on observe
Quelle est la répartition d'une variable numérique ?	Histogramme	Forme, concentration et asymétrie
Y a-t-il des valeurs atypiques ?	Boîte à moustaches	Médiane, quartiles et valeurs éloignées
Comment une valeur évolue-t-elle dans le temps ?	Courbe	Tendance, saisonnalité et ruptures
Comment comparer des catégories ?	Barres	Effectifs, sommes ou moyennes par catégorie
Deux variables numériques évoluent-elles ensemble ?	Nuage de points	Relation, groupes et valeurs atypiques

Un graphique ne doit pas seulement être « joli ». Son titre doit préciser la mesure, l'unité et la période ; ses axes doivent être lisibles ; la source et les filtres importants doivent être connus. Pour comparer des catégories, l'effectif et la moyenne ne racontent pas la même histoire : une catégorie peut avoir une moyenne élevée avec très peu d'observations.

6. Profilage automatique des données

Sur un nouveau jeu de données, il est souvent utile d'obtenir rapidement une vue d'ensemble avant de se lancer dans une exploration manuelle détaillée.

! Zoom sur les tendances : rapports d'exploration automatisés

Des bibliothèques comme `ydata-profiling` (anciennement `pandas-profiling`) ou `Sweetviz` génèrent, en une seule instruction, un rapport HTML complet : statistiques descriptives par variable, distributions, valeurs manquantes, corrélations entre variables, alertes de qualité (variables constantes, fortement corrélées, etc.).

```
from ydata_profiling import ProfileReport

profil = ProfileReport(df, title="Exploration du jeu de données")
profil.to_file("rapport_exploration.html")
```

Ces outils ne remplacent pas le raisonnement de l'analyste — ils accélèrent la première prise de contact avec un jeu de données et aident à repérer rapidement les points nécessitant une attention particulière, avant une exploration ciblée avec les techniques présentées dans ce chapitre.

Exercices

Exercice 0.0.1 (Interpréter des statistiques descriptives). Pour une variable `revenu_mensuel` (en MAD), on obtient : moyenne = 8 500, médiane = 6 200, écart-type = 5 400, asymétrie (*skewness*) = 2,3.

- Que suggère l'écart important entre la moyenne et la médiane ?
- Que confirme la valeur positive et élevée de l'asymétrie ?

Exercice 0.0.2 (Choisir le bon coefficient de corrélation). Pour chacune des situations suivantes, indiquez s'il est préférable d'utiliser Pearson ou Spearman :

- La relation entre l'ancienneté d'un employé (en années) et son salaire, globalement croissante mais pas forcément linéaire.
- La relation entre la taille et le poids d'un ensemble d'individus adultes, approximativement linéaire.

Exercice 0.0.3 (Construire un plan d'exploration). On dispose d'un jeu de données de transactions d'un site e-commerce marocain, avec les colonnes suivantes : `date_commande`, `categorie_produit`, `prix`, `region_client`, `moyen_paiement`.

Proposez, pour ce jeu de données, un exemple concret d'analyse univariée, un exemple d'analyse bivariée et un exemple d'analyse multivariée, en vous appuyant sur ces colonnes.

Exercice 0.0.4 (Calculer des indicateurs simples). Les durées de livraison, en jours, sont : 2, 3, 3, 4, 8.

- Calculez la moyenne, la médiane et l'étendue.
- Quel indicateur est le moins influencé par la durée de 8 jours ?
- Quelle question supplémentaire poseriez-vous avant de conclure que le service est lent ?

Exercice 0.0.5 (Lire une dispersion). Deux classes ont la même moyenne à un examen : 12/20. La classe A a un écart-type de 1,5 et la classe B un écart-type de 4.

- Quelle classe est la plus homogène ?
- Que signifie l'écart-type plus élevé de la classe B ?
- Peut-on dire que les étudiants de B ont de moins bonnes notes ?

Exercice 0.0.6 (Choisir le bon graphique). Associez chaque besoin à un graphique : histogramme, courbe, barres, boîte à moustaches ou nuage de points.

- Comparer le chiffre d'affaires de quatre régions.
- Observer le nombre de commandes chaque mois.
- Repérer des valeurs extrêmes dans les salaires.
- Étudier le lien entre surface et prix d'un logement.

Exercice 0.0.7 (Corrélation et causalité). Dans une ville, le nombre de glaces vendues et le nombre de baignades augmentent tous les deux en été.

- Quelle relation statistique peut-on observer ?
- Pourquoi ne peut-on pas conclure que les glaces provoquent les baignades ?
- Quelle variable cachée peut expliquer les deux évolutions ?

Exercice 0.0.8 (Interpréter un tableau agrégé). Un tableau par région présente les résultats suivants :

Région	Nombre de commandes	Montant moyen (MAD)
Nord	100	180
Centre	20	350
Sud	80	200

- Quelle région a le montant moyen le plus élevé ?
- Pourquoi faut-il également regarder le nombre de commandes ?
- Calculez le montant total estimé pour chaque région et identifiez la région au total le plus élevé.

Introduction à l'entrepôt de données

💡 Objectifs du chapitre

À la fin de ce chapitre, l'étudiant sera capable de :

- **Expliquer** pourquoi une organisation sépare souvent les systèmes opérationnels des données destinées à l'analyse ;
- **Distinguer** une base transactionnelle, un entrepôt de données, un lac de données et un datamart ;
- **Lire** un modèle en étoile composé d'une table de faits et de tables de dimensions ;
- **Définir** le grain d'une table de faits et choisir des mesures adaptées ;
- **Décrire** les grandes étapes d'un flux ETL/ELT, de la source au tableau de bord.

Un tableau de bord fiable ne commence pas par un graphique. Il commence par des données organisées, documentées et mises à jour de façon régulière. L'**entrepôt de données** (*data warehouse*) fournit cette base pour l'analyse.

1. Pourquoi un entrepôt de données ?

1.1. Les systèmes opérationnels

Une application de vente, une caisse ou un site web utilise une base **opérationnelle** pour enregistrer les événements du quotidien : créer une commande, modifier une adresse, enregistrer un paiement. Ces systèmes doivent répondre vite à de petites opérations et éviter les incohérences.

Une base opérationnelle est souvent organisée pour limiter la répétition des informations. Une commande peut être répartie entre `clients`, `commandes` et `lignes_commande`, reliées par des identifiants.

Cette organisation est adaptée à l'écriture, mais une question analytique comme « Quel est le chiffre d'affaires mensuel par région et par catégorie depuis trois ans ? » peut nécessiter de nombreuses jointures et plusieurs sources. Exécuter ce type de requête directement sur la base de production peut ralentir l'application.

1.2. La base analytique

Un **entrepôt de données** est un espace centralisé, organisé pour analyser l'historique de l'organisation. Il rassemble des données provenant de plusieurs sources, conserve leur contexte et les transforme dans un format plus simple à interroger.

Système	But principal	Exemple de question
Base opérationnelle (OLTP)	Enregistrer les événements courants	« Quel est le statut de la commande 1524 ? »
Entrepôt analytique (OLAP)	Comparer, agréger et suivre l'historique	« Comment les ventes ont-elles évolué par région ? »

Les deux systèmes sont complémentaires. L'entrepôt n'a pas pour objectif de remplacer l'application métier.

1.3. Entrepôt, lac et datamart

- Un **data warehouse** contient des données structurées, préparées et gouvernées pour l'analyse.
- Un **data lake** peut stocker des données brutes de formats variés (fichiers, textes, événements) avant leur usage ; il demande des règles claires pour rester compréhensible.
- Un **datamart** est une partie orientée vers un besoin ou un service, par exemple un datamart des ventes ou des ressources humaines.

 Un entrepôt ne corrige pas une mauvaise question

Centraliser beaucoup de données ne suffit pas. Il faut définir le vocabulaire métier, les unités, la fréquence de mise à jour et les indicateurs attendus. Sinon, on obtient plusieurs chiffres différents pour un même « chiffre d'affaires ».

2. Le modèle en étoile

Le modèle en étoile est une organisation pédagogique et professionnelle très fréquente pour les données analytiques. Il place une **table de faits** au centre et des **tables de dimensions** autour.

2.1. La table de faits

Une table de faits contient les événements mesurables : ventes, livraisons, appels ou visites. Elle comprend généralement :

- des clés vers les dimensions (`client_id`, `produit_id`, `date_id`, `magasin_id`) ;
- des mesures numériques (`quantite`, `montant`, `remise`) ;
- une ligne par événement selon un niveau de détail défini.

Le **grain** est la phrase qui précise ce que représente une ligne. Par exemple : « une ligne représente un produit d'une commande, dans un magasin, à une date donnée ». Le grain doit être défini avant de choisir les mesures.

2.2. Les tables de dimensions

Une dimension décrit le contexte d'un fait :

- `dim_date` : jour, mois, trimestre, année ;
- `dim_produit` : produit, catégorie, marque ;
- `dim_client` : ville, région, segment ;
- `dim_magasin` : magasin, ville, zone.

Les dimensions contiennent les attributs utilisés pour filtrer, regrouper et expliquer les mesures. Elles sont souvent dénormalisées pour rendre l'exploration plus simple.

```

      dim_date
      |
dim_client ---- fact_ventes ---- dim_produit
      |
      dim_magasin

```

2.3. Exemple de modèle de ventes

```
fact_ventes
-----
date_id
produit_id
client_id
magasin_id
quantite
prix_unitaire
remise
montant_net

dim_produit
-----
produit_id
libelle
categorie
marque

dim_date
-----
date_id
date_complete
mois
trimestre
annee
```

Le montant net peut être calculé par :

$$\text{montant_net} = \text{quantité} \times \text{prix unitaire} - \text{remise}.$$

Un modèle en étoile ne dicte pas un seul outil : il peut être utilisé dans une base SQL, un fichier analytique ou un modèle Power BI.

i Le rôle de la dimension Date

Même si la source contient déjà une date, une dimension Date facilite les regroupements par mois, trimestre, semaine ou jour ouvré. Elle permet aussi de définir clairement les périodes comparées.

3. Les mesures et les indicateurs

Une **mesure additive** peut être additionnée sur toutes les dimensions, comme le nombre d'articles ou le montant des ventes. Une mesure **semi-additive**, comme un stock de fin de journée, peut être additionnée par produit mais pas dans le temps. Une mesure **non additive**, comme un taux ou une moyenne, doit être recalculée à partir de ses éléments plutôt que sommée.

Exemple : si une région a 10 commandes et une autre 2 commandes, la moyenne globale ne doit pas être calculée comme la moyenne des deux moyennes si les effectifs sont différents. Il faut utiliser :

$$\text{moyenne globale} = \frac{\text{total des montants}}{\text{nombre total de commandes}}.$$

Les noms des indicateurs doivent préciser leur définition. « Ventes » peut signifier chiffre d'affaires brut, chiffre d'affaires net, commandes payées ou unités vendues. Dans un dictionnaire métier, on documente la formule, l'unité, la période et les filtres.

4. Le flux ETL/ELT

Les données passent généralement par un flux en plusieurs étapes :

1. **Extract (extraire)** : lire les sources opérationnelles, fichiers ou API ;
2. **Transform (transformer)** : convertir les dates, harmoniser les unités, dédupliquer, contrôler les valeurs et calculer les champs dérivés ;
3. **Load (charger)** : déposer les données préparées dans les tables de l'entrepôt.

Dans une approche **ELT**, on charge d'abord les données dans une plateforme capable de les traiter, puis on réalise les transformations à l'intérieur de cette plateforme. Les deux approches poursuivent le même but : rendre les données fiables et exploitables.

Un chargement quotidien doit pouvoir répondre à quatre questions simples : quelles sources ont été lues ? combien de lignes ont été reçues ? combien ont été rejetées ? quand le chargement s'est-il terminé ?

! Qualité et traçabilité

Conservez les données sources ou un journal de chargement, le nombre de lignes avant et après transformation, les erreurs détectées et la date de mise à jour. Ces informations permettent de comprendre pourquoi un indicateur a changé.

5. Gouvernance et sécurité, sans complexité inutile

La **gouvernance** désigne les règles qui rendent les données compréhensibles, accessibles aux bonnes personnes et utilisées de manière responsable. Pour commencer, un projet peut mettre en place :

- un dictionnaire des tables et des indicateurs ;
- un responsable pour chaque donnée importante ;
- des règles de qualité (unicité d'un identifiant, prix positif, date obligatoire) ;
- des droits d'accès adaptés, notamment pour les données personnelles ;
- une fréquence de mise à jour annoncée.

Un tableau de bord ne devrait pas exposer inutilement des noms, numéros de téléphone ou adresses. On peut agréger les résultats, masquer les colonnes sensibles et limiter les accès selon le rôle de l'utilisateur.

Exercices

Consigne

Les exercices vérifient la compréhension et peuvent être résolus sans ordinateur. Les calculs sont volontairement courts.

Exercice 0.0.1 (OLTP ou OLAP ?). Indiquez si la question relève plutôt d'un système opérationnel ou analytique :

- Modifier l'adresse d'un client.
- Comparer les ventes des quatre derniers trimestres par région.
- Vérifier si une commande est payée.
- Calculer la médiane des délais de livraison de l'année.

Exercice 0.0.2 (Identifier le grain). Une table `fact_ventes` contient une ligne par produit présent dans une commande. Formulez le grain avec une phrase et expliquez pourquoi une commande de trois produits peut produire trois lignes.

Exercice 0.0.3 (Faits ou dimensions). Classez les éléments suivants : `montant_net`, `categorie`, `annee`, `quantite`, `ville_client`, `prix_unitaire`.

Exercice 0.0.4 (Calculer une mesure). Une ligne indique 3 articles à 120 MAD chacun et une remise de 40 MAD. Calculez le montant net. Une deuxième ligne indique 2 articles à 80 MAD sans remise. Quel est le montant total des deux lignes ?

Exercice 0.0.5 (Mesure additive ou non ?). Indiquez si l'on peut additionner directement la mesure sur les périodes suivantes :

- Le montant de ventes.
- Le stock disponible à la fin de chaque jour.
- Un taux de conversion en pourcentage.

Exercice 0.0.6 (Comprendre ETL). Remettez dans l'ordre : charger les données préparées, extraire un fichier CSV, convertir les dates et contrôler les prix négatifs.

Exercice 0.0.7 (Moyenne globale). La région A compte 10 commandes de 100 MAD en moyenne. La région B compte 2 commandes de 300 MAD en moyenne.

- Quel est le total estimé de chaque région ?
- Calculez la moyenne globale.
- Pourquoi la moyenne des deux moyennes, 200 MAD, est-elle incorrecte ?

Exercice 0.0.8 (Qualité et sécurité). Proposez une règle de qualité et une mesure de protection pour chacune des situations :

- La colonne `prix` contient parfois des valeurs négatives.
- Un tableau de bord doit être partagé avec des responsables régionaux.

Business Intelligence avec Power BI et Taipy

💡 Objectifs du chapitre

À la fin de ce chapitre, l'étudiant sera capable de :

- **Définir** la Business Intelligence et distinguer un indicateur, un rapport et un tableau de bord ;
- **Construire** mentalement le parcours d'une donnée depuis la source jusqu'à une décision ;
- **Importer et préparer** des données dans Power BI avec Power Query ;
- **Comprendre** un modèle en étoile et créer des mesures simples en DAX ;
- **Choisir** entre Power BI et une application Python avec Taipy selon le besoin ;
- **Expliquer** les principes d'un tableau de bord lisible : objectif, public, filtres, contexte et action.

La **Business Intelligence (BI)** transforme des données préparées en informations régulièrement consultées pour piloter une activité. Elle ne consiste pas à accumuler des graphiques : un bon tableau de bord aide une personne précise à répondre à des questions précises et à décider.

1. Comprendre la BI

1.1. Du besoin à l'indicateur

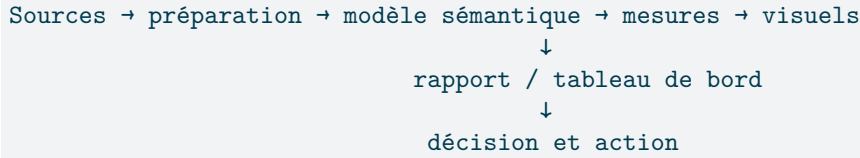
Un **indicateur** est une mesure définie par une formule, une période et une population. Un **KPI** (*Key Performance Indicator*) est un indicateur considéré comme essentiel pour suivre un objectif.

Exemple pour un site de livraison :

Élément	Exemple
Objectif	Livrer les commandes à temps
KPI	Taux de livraison à temps
Formule	livraisons à temps / livraisons terminées
Période	semaine ou mois
Cible	au moins 95 %
Action possible	examiner les villes sous la cible

Un rapport peut contenir plusieurs pages et analyses détaillées. Un tableau de bord est généralement plus synthétique : il met en évidence les quelques indicateurs et alertes nécessaires au suivi.

1.2. Le parcours d'une solution BI



Le modèle **sémantique** donne un sens commun aux tables, aux relations et aux mesures. Deux personnes doivent obtenir le même chiffre lorsqu'elles utilisent le même KPI.

1.3. Règles d'un tableau de bord débutant

Un tableau de bord lisible :

1. commence par le public et la décision attendue ;
2. affiche une période et une unité ;
3. présente quelques KPI importants avant les détails ;
4. utilise une couleur pour signaler une information, pas pour décorer chaque objet ;
5. permet d'explorer sans cacher la définition des indicateurs ;
6. indique la date de dernière actualisation.

2. Power BI : une démarche structurée

Power BI est une plateforme de Business Intelligence qui permet de connecter des sources, de les transformer, de créer un modèle de données, de calculer des mesures et de construire des rapports interactifs. Le parcours présenté ici commence dans Power BI Desktop ; le partage dépend ensuite de l'environnement et des droits de l'organisation.

2.1. Étape 1 — Importer les sources

On peut importer un fichier CSV, un classeur Excel, une base SQL ou d'autres sources disponibles dans l'environnement. À l'import, il faut vérifier :

- les noms de colonnes et leurs types ;
- le séparateur et l'encodage d'un CSV ;
- le type date, nombre ou texte ;
- le nombre de lignes et la période couverte ;
- la présence de doublons et de valeurs manquantes.

Importer n'est pas encore analyser : il faut comprendre ce qui a été chargé.

2.2. Étape 2 — Transformer avec Power Query

Power Query sert à préparer les données avant leur utilisation dans le modèle. On peut supprimer des colonnes inutiles, renommer un champ, convertir un type, filtrer des lignes, remplacer une valeur, fusionner des tables ou ajouter une colonne calculée.

Une transformation doit rester explicable et reproductible. Power Query conserve les étapes appliquées ; il vaut mieux créer plusieurs étapes nommées et simples qu'une modification manuelle impossible à refaire.

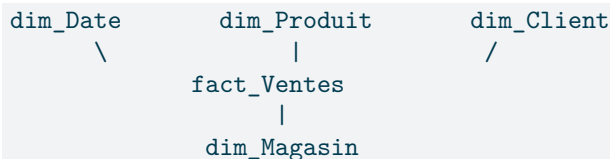
Exemples de contrôles :

- **Montant** est numérique et positif ;

- DateCommande est bien de type date ;
- ProduitID et DateID sont présents ;
- la catégorie vide est traitée explicitement ;
- le nombre de lignes final est comparé au nombre attendu.

2.3. Étape 3 — Construire le modèle

Pour un modèle de ventes, on privilégie un schéma en étoile :



La table de faits contient les lignes de vente et les dimensions permettent de filtrer par date, produit, client ou magasin. Les relations doivent être vérifiées : une dimension possède généralement une ligne par identifiant, et plusieurs lignes de faits peuvent lui être liées.

Le piège des relations ambiguës

Si une dimension contient plusieurs lignes pour le même identifiant, ou si deux tables sont reliées de manière inattendue, les totaux peuvent être multipliés. Vérifiez toujours le grain, les clés et un total connu avant de publier un rapport.

2.4. Étape 4 — Créer des mesures en DAX

DAX est le langage de formules utilisé dans les modèles Power BI. Une **mesure** est recalculée selon le contexte de filtre du visuel, de la page ou de l'utilisateur.

```

ChiffreAffaires = SUM(fact_Ventes[MontantNet])

NombreCommandes = DISTINCTCOUNT(fact_Ventes[CommandeID])

PanierMoyen = DIVIDE([ChiffreAffaires], [NombreCommandes])

```

DIVIDE évite une erreur lorsque le dénominateur est nul. Pour débiter, il faut distinguer une mesure d'une colonne calculée : la mesure s'adapte aux filtres, tandis qu'une colonne calculée est créée ligne par ligne dans la table.

Une mesure peut être comparée à une période précédente, mais la logique de dates doit être préparée soigneusement. Dans un premier tableau de bord, trois ou quatre mesures bien définies valent mieux que de nombreuses formules difficiles à expliquer.

2.5. Étape 5 — Construire les visuels

Pour une page de suivi des ventes, une composition simple peut être :

- une ligne de cartes KPI : chiffre d'affaires, commandes, panier moyen, taux de livraison à temps ;
- une courbe du chiffre d'affaires par mois ;
- un graphique en barres par région ou catégorie ;

- un tableau détaillé filtrable ;
- des segments (*slicers*) pour l'année, la région et la catégorie.

Chaque visuel doit répondre à une question. Un graphique avec trop de séries ou trop de couleurs devient difficile à lire.

2.6. Étape 6 — Vérifier et partager

Avant tout partage, vérifiez les totaux sur un petit exemple connu, la date d'actualisation, les filtres, les titres et les droits d'accès. Un rapport partagé avec des données personnelles doit respecter les règles de l'organisation et le principe du moindre accès.

3. Taipy : créer une application BI en Python

Taipy permet de construire des applications Python interactives avec des contrôles, des graphiques, des tableaux et des filtres. Il est particulièrement intéressant quand l'équipe veut intégrer l'analyse dans une application métier ou conserver une logique Python personnalisée.

3.1. Quand choisir Taipy ?

Taipy peut être pertinent lorsque :

- les traitements sont déjà écrits en Python ;
- l'application doit combiner un tableau de bord avec une logique métier ;
- l'on veut contrôler l'interface et le déploiement comme un projet logiciel ;
- l'utilisateur doit modifier un paramètre et observer immédiatement un résultat.

Power BI peut être plus adapté lorsqu'une organisation veut une solution de BI prête à l'emploi, un catalogue de rapports, des connexions gérées et des usages majoritairement orientés vers l'exploration métier.

Ce ne sont pas des outils absolument opposés. Une équipe peut préparer les données avec Python, les exposer dans Power BI, ou intégrer un indicateur Power BI dans un parcours applicatif selon son architecture.

3.2. Un exemple Taipy minimal

Le code suivant illustre le principe avec des données déjà disponibles. Il ne remplace pas la préparation et la validation des données.

```
import pandas as pd
from taipy.gui import Gui

df = pd.DataFrame({
    "Mois": ["Janvier", "Février", "Mars"],
    "Ventes": [12000, 14500, 13200],
})

mois = "Tous"
texte = "### Suivi des ventes"

page = """
# Suivi des ventes
```

```
Sélection : <|{mois}|selector|lov=Tous;Janvier;Février;Mars|>

<|{df}|table|>
"""

Gui(page=page).run()
```

Dans cet exemple, `df` contient les données affichées, `page` décrit l'interface et le sélecteur permet de choisir une valeur. Dans un vrai projet, on ajoute une fonction qui filtre les données, des indicateurs calculés et des graphiques adaptés. Les noms exacts de certains contrôles peuvent évoluer selon la version de Taipy : il faut vérifier la documentation de la version utilisée.

3.3. Organiser une petite application

Même une application pédagogique gagne à séparer :

```
projet/
  data/           # données sources ou exemple
  preparation.py  # lecture et nettoyage
  indicateurs.py  # calculs métier
  app.py          # interface Taipy
  README.md       # installation et utilisation
```

Cette séparation permet de tester les indicateurs indépendamment de l'interface. L'application ne doit pas recalculer silencieusement un indicateur avec une définition différente de celle du rapport.

4. Comparaison synthétique

Critère	Power BI	Taipy
Profil	Plateforme BI avec interface graphique	Bibliothèque/applications Python
Préparation	Power Query et connexions	Python, pandas ou autres outils
Mesures	DAX et modèle sémantique	Fonctions et code Python
Usage principal	Rapports, exploration et partage BI	Application interactive intégrée à un projet
Point d'attention	Relations, contexte des mesures, droits	Organisation du code, déploiement, versions

Le choix doit partir du besoin, pas de la préférence pour un outil. Dans les deux cas, les mêmes bases restent indispensables : question métier, qualité des données, modèle clair, indicateurs documentés et visualisation lisible.

Exercices

i Consigne

Ces exercices sont conceptuels ou demandent seulement un calcul court. Aucun logiciel n'est nécessaire.

Exercice 0.0.1 (Indicateur ou action?). Pour l'objectif « réduire les retards de livraison », proposez un KPI, sa formule et une action possible si sa valeur diminue.

Exercice 0.0.2 (Lire un parcours BI). Associez chaque étape à son rôle : préparer, modéliser, visualiser, décider.

- Relier les ventes à la dimension Date.
- Convertir une colonne texte en date.
- Afficher les ventes par mois.
- Renforcer un transporteur après avoir constaté des retards.

Exercice 0.0.3 (Calculer un KPI). Sur 250 livraisons terminées, 225 ont été livrées à temps. Calculez le taux de livraison à temps et indiquez s'il atteint une cible de 95 %.

Exercice 0.0.4 (Power Query ou DAX?). Indiquez l'outil le plus adapté :

- Corriger le type d'une colonne Date.
- Calculer le chiffre d'affaires selon les filtres du rapport.
- Supprimer une colonne inutilisée avant le modèle.

Exercice 0.0.5 (Vérifier une relation). Une dimension **Produit** contient deux lignes pour le même **ProduitID**. Quel risque cela crée-t-il lorsqu'on relie cette dimension à une table de ventes? Quelle vérification simple proposeriez-vous?

Exercice 0.0.6 (Choisir un outil). Une équipe possède déjà plusieurs fonctions Python et veut intégrer un indicateur interactif dans une application interne. Une autre équipe veut surtout créer rapidement des rapports métiers partagés. Quel outil recommander à chacune, et pourquoi?

Exercice 0.0.7 (Lire une mesure). Le chiffre d'affaires est de 48 000 MAD pour 300 commandes.

- Calculez le panier moyen.
- Pourquoi faut-il utiliser le nombre de commandes et non le nombre de lignes de produits si le KPI est défini par commande?

Exercice 0.0.8 (Critiquer une page). Une page affiche 25 graphiques sans unité, sans date de mise à jour et avec une couleur différente pour chaque barre. Citez trois améliorations prioritaires et justifiez-les.

Cas pratique complet : analyser les ventes d'une petite chaîne de magasins

💡 Objectifs du chapitre

À la fin de ce cas pratique, l'étudiant sera capable de :

- **Cadrer** une question d'analyse à partir d'un besoin métier ;
- **Repérer** les problèmes de qualité dans un petit jeu de données ;
- **Calculer et interpréter** des indicateurs simples : chiffre d'affaires, quantité, panier moyen, médiane et taux ;
- **Choisir** un modèle en étoile adapté à une analyse de ventes ;
- **Proposer** une page Power BI et une vue Taipy cohérentes avec les questions ;
- **Rédiger** une conclusion prudente, fondée sur les données et accompagnée de limites.

Ce chapitre suit un projet de bout en bout. L'entreprise fictive **Atlas Marchés** possède trois magasins à Casablanca, Rabat et Tanger. La direction souhaite comprendre les ventes du premier trimestre 2026 et décider où concentrer ses efforts commerciaux. Les données sont volontairement petites afin que les calculs puissent être vérifiés à la main.

1. Le besoin métier

La direction pose quatre questions :

1. Quel est le chiffre d'affaires net par mois et par magasin ?
2. Quels produits ou catégories contribuent le plus aux ventes ?
3. Le panier moyen est-il différent selon les magasins ?
4. Quelles informations faudrait-il surveiller dans un tableau de bord mensuel ?

La question n'est pas « faire un graphique ». Le livrable attendu est une analyse descriptive, quelques explications prudentes et une proposition de suivi.

1.1. Périmètre et définitions

Le périmètre est limité aux ventes **payées** enregistrées entre le 1er janvier et le 31 mars 2026. Une ligne du fichier correspond à une ligne de produit dans une commande. Le montant net est défini ainsi :

$$\text{montant net} = \text{quantité} \times \text{prix unitaire} - \text{remise}.$$

Le **panier moyen** est :

$$\text{panier moyen} = \frac{\text{chiffre d'affaires net}}{\text{nombre de commandes distinctes}}.$$

Ces définitions sont annoncées avant les calculs ; elles évitent de changer de formule au milieu de l'analyse.

2. Le jeu de données

Voici un extrait pédagogique. Dans un vrai projet, le fichier complet pourrait contenir beaucoup plus de lignes.

Ligne	Commande	Date	Magasin	Catégorie	Produit	Qté	Prix unitaire	Remise	Statut
1	C001	05/01/2026	Casablanca	Épicerie	Huile	2	75	0	Payée
2	C001	05/01/2026	Casablanca	Épicerie	Riz	3	20	0	Payée
3	C002	08/01/2026	Rabat	Boissons	Jus	4	18	5	Payée
4	C003	15/01/2026	Tanger	Hygiène	Savon	5	12	0	Payée
5	C004	03/02/2026	Casablanca	Épicerie	Huile	1	75	0	Payée
6	C005	10/02/2026	Rabat	Épicerie	Riz	5	20	10	Payée
7	C006	18/02/2026	Tanger	Boissons	Jus	2	18	0	Annulée
8	C007	22/02/2026	Casablanca	Hygiène	Savon	6	12	0	Payée
9	C008	06/03/2026	Rabat	Boissons	Jus	6	18	0	Payée
10	C009	12/03/2026	Tanger	Épicerie	Riz	2	20	0	Payée
11	C010	20/03/2026	Casablanca	Boissons	Jus	3	18	5	Payée
12	C010	20/03/2026	Casablanca	Hygiène	Savon	2	12	0	Payée

2.1. Première lecture

Le grain est **une ligne de produit dans une commande**. La commande C001 apparaît deux fois car elle contient deux produits. Pour compter les commandes, il faut compter les identifiants distincts, et non les lignes.

La ligne C006 est annulée. Si le chiffre d'affaires concerne les ventes payées, elle doit être exclue, mais cette décision doit être documentée. Elle ne doit pas être supprimée du fichier original : on conserve le statut pour pouvoir analyser les annulations.

3. Préparer les données

3.1. Dictionnaire minimal

Colonne	Type attendu	Règle
Commande	Texte	Identifiant non vide
Date	Date	Date comprise dans la période
Magasin	Catégorie	Casablanca, Rabat ou Tanger
Qté	Entier	Strictement positive pour une vente
Prix unitaire	Décimal	Positif, en MAD
Remise	Décimal	Supérieure ou égale à 0, en MAD
Statut	Catégorie	Payée ou Annulée

Il faut également vérifier que la remise ne dépasse pas le montant brut de la ligne, que les dates ne sont pas inversées et que les identifiants ne comportent pas d'espaces cachés.

3.2. Transformation

Pour chaque ligne, on calcule :

```
montant_brut = quantité × prix_unitaire
montant_net = montant_brut - remise
mois = mois(date)
```

Puis on filtre `Statut = Payée` pour les indicateurs de chiffre d'affaires. On conserve une copie de l'ensemble initial et on compte les lignes exclues.

Pour l'extrait, 11 lignes sur 12 sont payées. La ligne annulée représente 1 ligne sur 12, soit environ 8,3 % des lignes ; ce pourcentage ne doit pas être interprété comme un taux de commandes annulées, car une commande peut comporter plusieurs lignes.

4. Calculer les indicateurs

4.1. Montants ligne par ligne

Sur les lignes payées :

Commande	Produit	Calcul	Montant net
C001	Huile	$2 \times 75 - 0$	150
C001	Riz	$3 \times 20 - 0$	60
C002	Jus	$4 \times 18 - 5$	67
C003	Savon	$5 \times 12 - 0$	60
C004	Huile	$1 \times 75 - 0$	75
C005	Riz	$5 \times 20 - 10$	90
C007	Savon	$6 \times 12 - 0$	72
C008	Jus	$6 \times 18 - 0$	108
C009	Riz	$2 \times 20 - 0$	40
C010	Jus	$3 \times 18 - 5$	49
C010	Savon	$2 \times 12 - 0$	24

Le chiffre d'affaires net de l'extrait est donc :

$$150 + 60 + 67 + 60 + 75 + 90 + 72 + 108 + 40 + 49 + 24 = \mathbf{795 \text{ MAD.}}$$

Mois	Chiffre d'affaires net	Commandes distinctes	Panier moyen
------	------------------------	----------------------	--------------

4.2. Par mois

Mois	Chiffre d'affaires net	Commandes distinctes	Panier moyen
Janvier	337	3	112,33
Février	237	3	79,00
Mars	221	3	73,67
Total	795	9	88,33

Le panier moyen total est $795/9 = 88,33$ MAD. Il ne faut pas faire la moyenne simple des trois paniers mensuels sans vérifier les effectifs ; ici ils sont égaux, mais ce n'est pas toujours le cas.

4.3. Par magasin et par catégorie

En regroupant les lignes payées :

Magasin	Chiffre d'affaires net	Commandes distinctes	Panier moyen
Casablanca	435	4	108,75
Rabat	157	3	52,33
Tanger	100	2	50,00

Catégorie	Chiffre d'affaires net
Épicerie	415
Boissons	224
Hygiène	156

Casablanca contribue le plus au chiffre d'affaires et possède le panier moyen le plus élevé dans cet extrait. Cela ne suffit pas à conclure que le magasin est le plus performant : il faudrait connaître le nombre de visiteurs, les coûts, la taille des magasins et une période plus longue.

4.4. Moyenne et médiane des montants de lignes

Les montants des lignes payées sont :

60, 60, 67, 72, 75, 90, 108, 24, 40, 49, 150

Triés :

24, 40, 49, 60, 60, 67, 72, 75, 90, 108, 150.

La médiane est la sixième valeur, soit **67 MAD**. La moyenne est $795/11 \approx 72,27$ MAD. La moyenne un peu supérieure à la médiane suggère que quelques lignes élevées, notamment 150 MAD, tirent la moyenne vers le haut.

5. Construire le modèle analytique

À partir de ce fichier, un modèle en étoile simple pourrait être :

```
dim_date -----\
dim_produit ----- fact_ventes
dim_magasin -----/
```

fact_ventes contient Commande, DateID, ProduitID, MagasinID, Quantite, PrixUnitaire, Remise, MontantNet et Statut. Les dimensions décrivent les dates, produits et magasins.

Les mesures attendues sont :

```
ChiffreAffaires = somme de MontantNet pour les lignes payées
Commandes = nombre d'identifiants Commande distincts pour les lignes payées
PanierMoyen = ChiffreAffaires / Commandes
Quantités = somme de Quantite pour les lignes payées
```

Le tableau de bord ne doit pas calculer le panier moyen comme la moyenne des montants de lignes : une commande peut contenir plusieurs lignes. Le grain explique cette différence.

6. Concevoir la page Power BI

Une page « Vue générale » peut contenir :

1. quatre cartes : chiffre d'affaires net, commandes, panier moyen et quantités vendues ;
2. une courbe du chiffre d'affaires par mois ;
3. des barres du chiffre d'affaires par magasin ;
4. des barres du chiffre d'affaires par catégorie ;
5. des segments pour le mois, le magasin et la catégorie ;
6. une note indiquant « données payées, période janvier-mars 2026, dernière actualisation ».

Une deuxième page « Diagnostic » peut détailler les produits, les annulations et les différences entre magasins. La première page répond à « où en sommes-nous ? », la seconde aide à explorer « où regarder ensuite ? ».

Avant le partage, on compare les cartes aux totaux calculés manuellement : 795 MAD, 9 commandes distinctes et 88,33 MAD de panier moyen.

7. Construire une vue Taipy

La même analyse peut être intégrée à une application Python. La logique doit être séparée de l'affichage :

```
def indicateurs(df):
    payees = df[df["Statut"] == "Payée"]
    chiffre_affaires = payees["MontantNet"].sum()
    commandes = payees["Commande"].nunique()
    panier_moyen = chiffre_affaires / commandes if commandes else 0
    return {
        "chiffre_affaires": chiffre_affaires,
```

```
"commandes": commandes,  
"panier_moyen": panier_moyen,  
}
```

Une interface Taipy peut ensuite proposer un filtre **Magasin**, afficher les trois indicateurs et mettre à jour une table mensuelle. Le résultat doit rester cohérent avec le dictionnaire métier et les calculs Power BI.

8. Interpréter et communiquer

Une conclusion raisonnable pour l'extrait serait :

Sur les ventes payées de l'extrait, le chiffre d'affaires net atteint 795 MAD pour 9 commandes, avec un panier moyen de 88,33 MAD. Casablanca représente 435 MAD et le panier moyen y est supérieur à celui de Rabat et Tanger. Les lignes de vente ont une médiane de 67 MAD et une moyenne de 72,27 MAD, ce qui signale une légère influence de quelques montants élevés. Ces résultats sont exploratoires : l'échantillon est très petit, ne couvre que trois mois et ne permet pas d'expliquer les causes ni de mesurer la rentabilité.

La conclusion sépare les **faits calculés**, les **interprétations prudentes** et les **limites**. Elle ne transforme pas une différence observée en causalité.

Exercices

i Consigne

Répondez sans ordinateur. Les données de l'énoncé suffisent ; les calculs sont courts et peuvent être réalisés avec une calculatrice.

Exercice 0.0.1 (Définir le grain). Quelle phrase décrit le grain du fichier ? Pourquoi la commande C001 occupe-t-elle deux lignes ?

Exercice 0.0.2 (Exclure une annulation). Pourquoi la ligne C006 n'est-elle pas incluse dans le chiffre d'affaires payé ? Peut-on la supprimer du fichier original ?

Exercice 0.0.3 (Calculer une ligne). Calculez le montant net de 4 jus à 18 MAD avec une remise de 5 MAD, puis celui de 5 savons à 12 MAD sans remise.

Exercice 0.0.4 (Vérifier le total). Le chiffre d'affaires net est de 795 MAD et le magasin de Casablanca réalise 435 MAD.

- Quelle part du chiffre d'affaires total représente Casablanca ?
- Quel montant reste pour Rabat et Tanger ensemble ?

Exercice 0.0.5 (Commandes et lignes). Il y a 11 lignes payées mais seulement 9 commandes distinctes.

- Pourquoi ces deux nombres sont-ils différents ?
- Quel nombre utiliser pour calculer le panier moyen ?

Exercice 0.0.6 (Comparer moyenne et médiane). La moyenne des montants de lignes est 72,27 MAD et la médiane est 67 MAD. Que suggère cette différence ? Quelle valeur de la liste contribue particulièrement à tirer la moyenne vers le haut ?

Exercice 0.0.7 (Choisir les indicateurs du tableau de bord). Parmi les éléments suivants, lesquels doivent apparaître en priorité sur la page générale : chiffre d'affaires net, identifiant de chaque client, commandes distinctes, panier moyen, une liste de 25 couleurs, date d'actualisation ? Justifiez les choix.

Exercice 0.0.8 (Limites et décisions). La direction demande : « Casablanca est-elle le meilleur magasin ? »

- a. Donnez une conclusion que les données permettent de soutenir.
- b. Citez deux informations manquantes avant de conclure sur la performance globale.

References

Knafllic, Cole. 2015. *Storytelling with Data*. Wiley.

McKinney, Wes. 2022. *Python for Data Analysis*. O'Reilly Media.